
Partial Verifiability Is Enough: Bootstrapping a Fully-Local Knowledge-Graph Question-Answering Planner Past Its Cloud Teachers

Anonymous Author

Department of Electronic and Electrical Engineering
University College London

uceeh01@ucl.ac.uk

Abstract

Question answering over structured public data faces a trust dilemma. Large language models can read natural-language questions but answer unverifiably, while symbolic executors answer verifiably but cannot read. This dissertation presents CICADA, a neuro-symbolic system over a knowledge graph of 215,221 UK public-procurement contract awards. In CICADA the language model only proposes; a deterministic ground-compile-execute-verify stack is the sole answer authority. The central claim is one sentence: *supervision flows only from the verifiable region, yet the learned competence extends beyond it*. All training data is distilled through the pipeline’s own verifier and an independent oracle that filters but never authors. Abstention closes the verifier’s blind region: three abstention classes are first-class benchmark citizens, and anti-hallucination supervision flows through the same verifier gate. The bootstrapped system runs entirely on a single local GPU, with both the understanding and planning stages served as 8B LoRA adapters. It outperforms its cloud-hybrid counterpart on the held-out test set (n=2,285: 85.65% vs 78.03%; +7.61pt, 95% CI [+6.10, +9.13], McNemar $p < 10^{-15}$). It also exceeds its cloud teacher by +15.89pt (69.76% → 85.65%; CI [+14.07, +17.70], $p < 10^{-15}$; single teacher replicate), at zero marginal API cost. Robustness of the teacher gap to provider nondeterminism was checked on the development set, where the student beats each of three teacher replicates (+35/-5, +34/-4, +30/-5; $p \leq 2e-5$). De-teaching replicates on a second base (Llama fully-local 83.33% vs its hybrid 75.97%; +7.35pt, CI [+5.73, +8.97]); all five pairings hold at $p < 10^{-14}$. Mechanistically, across three teacher replicates accuracy decays 6–12 points from the iid to the hard-composite slice. The fully-local student shows no measurable decay (+0.8pt, n=136) with the same planning checkpoint, which we attribute to the distilled understanding stage. All results rest on a 12,828-question benchmark with fully parameterised gold programs, whose oracles are validated by an independent second implementation at 99.88% agreement.

1 Introduction

2 Task and motivation

Every year the United Kingdom publishes hundreds of thousands of contract-award notices. These notices describe how public money is spent: which authority bought what, from which supplier, for how much, and under which procurement category. This dissertation studies natural-language question answering over a knowledge graph built from 215,221 such contract-award records (artifact-verified count; results table, promoted rows). A journalist, auditor, or council officer should be able to ask "Which supplier received the most cleaning contracts from London boroughs in 2022?". They should receive either an answer whose every step is traceable to specific records in the graph, or an explicit refusal to answer when the question is ambiguous, unsupported by the schema, or matched by no records at all.

In this domain the second half of that requirement matters as much as the first. Questions about public spending carry an accountability weight that most question-answering settings do not. A fluent but fabricated total, or a confidently named supplier who never held the contract, is worse than no answer. Such an answer can flow into reporting and decision-making with the borrowed authority of the underlying official data. The system studied here, CICADA, is therefore built around a single non-negotiable constraint: an answer is only ever produced by deterministic execution against the knowledge graph. Where the system cannot support an answer, it must abstain.

Large language models are the obvious choice for the language-understanding half of this task. The largest cloud-hosted models can indeed perform it to a useful standard. However, using a cloud model on public-sector data raises three linked objections. It is costly, because every query adds marginal API cost that grows with usage instead of amortising. It is uncontrollable, because the provider may change, deprecate, or nondeterministically vary the model under a fixed application; we measure this variability directly in this work. It is also non-private, because the queries, and any internal data joined against them, must leave the organisation's infrastructure. A local system built from small open-weight models would resolve all three objections at once, if it could be made competent enough.

3 The trust dilemma, and why retrieval augmentation is not enough

The task therefore sits on a trust dilemma. Language models can read the question but answer unverifiably: nothing in a free-form generated response separates a genuine aggregation over the data from a plausible confabulation. Query executors answer verifiably but cannot read: a database engine will faithfully compute whatever it is asked, but turning an ambiguous English question into the right query is exactly the hard part. Any credible architecture must couple the two, and trust is either established or quietly lost at the coupling point.

The most common coupling today is retrieval-augmented generation (RAG). Relevant records are retrieved into the model's context window, and the model composes an answer over them. For this task RAG fails structurally, not incidentally. Procurement questions are dominated by exhaustive computation: counts, sums, rankings, and multi-hop joins whose correct answer depends on *all* matching records, not a representative sample. No top-k retrieval window can hold "every contract awarded by every London borough in 2022". A language model asked to aggregate over a truncated window will produce an answer that is wrong in a way no reader can detect. Our measurements support this. On our 260-question development set under the v1 pipeline, a naive RAG baseline achieved 31.5% (82/260) and a strengthened RAG baseline 28.8% (75/260). Under the same v1 pipeline and on the same development set, a completely untrained 8B model, given only the executable-plan scaffolding described in Chapter 5, reached 61.5% (160/260). (Under the revised v2.2 pipeline this zero-shot scaffolding floor rises to 70.4% on the same development set.) The lesson is that the scaffolding, which forces the model to emit a checkable, executable plan rather than a free-form answer, is worth more than retrieval before any training has taken place.

Scaffolding alone, however, still leaves a gap of ten to twenty accuracy points against what the task demands. Closing that gap is where a second dilemma appears. The classical route to a competent small model is distillation from a large one: the cloud teacher labels training data, and the student imitates. But classical distillation assumes a *trusted* teacher, and on this task the teacher is not trustworthy. As Chapter 6 shows, the cloud teacher pipeline itself answers only around seven in ten held-out questions correctly. Imitating an untrusted teacher transfers its errors along with its competence. The central move of this dissertation is to show that teacher trust can be replaced by something the task itself provides: *partial verifiability*. A large and precisely characterisable region of the failure space is mechanically checkable. The checks ask whether a plan compiles, whether its entities ground in the graph, whether it executes, whether its structure matches its stated intent, and, at training time only, whether its answer agrees with an independent oracle. Supervision can be drawn only from that region, discarding everything the checks cannot vouch for, regardless of which model produced it.

4 The one-sentence thesis

The dissertation defends a single sentence:

Supervision flows only from the verifiable region, yet the learned competence extends beyond it.

The first clause is a statement of construction. Every training example used anywhere in this work has passed a deterministic compile-ground-execute-verify chain. During training-data harvest only, it has also passed an agreement check against an independently computed oracle answer. The oracle *filters* candidate examples; it never *authors* them: no gold plan or gold answer is ever copied into the training data. Whatever cannot be projected onto structure and execution receives no supervision at all, because no mechanical check can certify it. This residue is chiefly the purely semantic question of whether a plan means what the question means.

The second clause is the empirical surprise, and the mechanism behind the headline results. If the student merely memorised the verifiable region, its competence should stop at the projection line. It should perform well where the checks reach and collapse where they do not. It does not. We contrast a development-set i.i.d. slice (n=103) with a *hard-composite* slice (n=136). The hard-composite slice combines hard operators, second-level paraphrase, and abstention cases; this difficulty composite appears in both training and test with disjoint plans and novel surfaces. On this contrast the cloud teacher’s accuracy decays by between 6 and 12 points across three replicates (−10.3, −6.1, −12.5), while the fully-local student shows no measurable decay (+0.8 points, n=136). The student trained only on verifier-passed data generalises *past* the region the verifier can see, where the far larger teacher degrades. This is the mechanism answer to the question "why is partial verifiability enough": the capability learned from verifiable supervision is wider than the coverage of the signal that taught it.

5 Approach overview: the three-layer structure

The system design that carries this thesis has three layers. Chapter 5 treats them fully; we summarise them here.

The hard-verification layer. Every property along the semantic chain from question to answer that *can* be projected onto structure or execution *is* projected, and pinned by a deterministic check. The executor’s behaviour is pinned to the plan graph by compilation and execution. The plan graph is pinned to the intermediate briefing by structural comparison in code. Slots, entities, literals, numbers, roles, and cue words are cross-checked between briefing and question. We state this precisely: these checks do not verify everything, and this dissertation never claims that they do. Everything hard-verifiable is hard-verified, and the residue is squeezed to a minimum. The checks shrink the blind region layer by layer; they never eliminate it.

The abstention layer. The unprojectable residue, chiefly whether the plan’s meaning matches the question’s meaning, forms a blind region. The system’s answer to the blind region is calibrated refusal. Abstention is a first-class output. The benchmark contains three families of abstention traps (ambiguous questions, schema-unsupported questions, empty-result questions), and these serve as the completeness test of this layer rather than as decoration.

The learning layer. Signal flowing out of the verifiable region drives distillation and bootstrapping. Teacher outputs are filtered rather than trusted, student outputs are self-harvested through the same filter, and the filtered pool trains both stages of the stack. The learned competence then covers part of the blind region, which is the one-sentence thesis restated as an engineering loop.

6 Contributions

The dissertation makes four contributions, referred to throughout as the four pillars.

Table 1: Contributions of this dissertation: the four evidence pillars supporting the partial-verifiability thesis.

System	Final-test accuracy
Teacher (cloud, 1 replicate)	69.76% (1594/2285)
Llama-3.1-8B hybrid	75.97%
Qwen3-8B hybrid	78.03%
Llama-3.1-8B fully local	83.33%
Qwen3-8B fully local	85.65% (1957/2285)

Pillar 1 — the bootstrap closed loop (Chapter 6). Three instantiations of learning from the verifiable region. We filter an untrusted cloud teacher’s outputs into clean supervision. We run a self-harvest round in which the student, generating and filtering its own training data, overtakes the teacher that taught it. We then apply a denoising distillation to the first pipeline stage (understanding/briefing), which lets the entire stack run locally. Together these steps bootstrap both stages of a neuro-symbolic system past both of its cloud teachers, over a 12,828-question benchmark (9,267 train / 556 dev-tune / 671 dev-select / 49 dev-smoke / 2,285 final test).

Pillar 2 — gain decomposition (Chapters 5–6). A separation of the hard-verification layer’s contribution from the learning layer’s. The zero-shot scaffolding floor measures what deterministic checking and repair buy before any training. The training ladder above it measures what verifier-filtered learning adds. The development-set ladder establishes that the decomposition exists; all confirmatory claims are carried by the final test set.

Pillar 3 — DPO near-miss toxicity (Chapter 7). A mechanism-explained negative result. Using verifier-passing-but-wrong outputs as preference-learning negatives seems natural, but it can suppress the very competencies the positives teach. The contrast between subtractive preference suppression and additive distillation explains when each is safe.

Pillar 4 — benchmark methodology (Chapter 4). The abstention layer’s test apparatus and the credibility foundation under every number in this dissertation. It uses dual-implementation oracles whose agreement is measured rather than assumed, and plan-level split isolation so that no test plan is seen in training. It also follows a strict development/confirmatory discipline in which model selection and pipeline fixes are derived only from development data.

7 Results preview

The confirmatory evaluation is a five-system pairing on the held-out final test set ($n=2,285$, v4.1 benchmark, evaluation protocol with repair budget 1). It covers the cloud teacher, two hybrid students (cloud understanding stage, local planning stage), and two fully-local students (both stages local 8B LoRA adapters), on two independent base models.

Every adjacent and non-adjacent pairing in this scoreboard is significant at McNemar $p < 10^{-14}$. The headline result is that the fully-local Qwen3-8B stack exceeds its own cloud teacher by **+15.89 points** (95% CI [+14.07, +17.70], discordant pairs +406/−43, $p < 10^{-15}$). The result replicates on a second base model: the fully-local Llama-3.1-8B stack reaches 83.33% (+13.57 points over the teacher, CI [+11.81, +15.32]). Because the teacher was run as a single replicate on the final test set, we check the robustness of the student–teacher gap to provider nondeterminism separately on the development set. There the student beats each of three independent teacher replicates (discordant pairs +35/−5, +34/−4, +30/−5; $p \leq 2e-5$). The fully-local stack achieves this at zero marginal API cost, on a single local GPU, with no data leaving the machine.

8 Dissertation structure

Chapter 2 surveys the four literatures the work sits between, and draws the dividing line against its nearest neighbours on filter strength and blind-region handling. Chapter 3 describes the procurement data and

the construction of the knowledge graph. Chapter 4 presents the benchmark and its integrity apparatus (Pillar 4). Chapter 5 presents the three-layer system and the deterministic check chain, and measures the scaffolding floor (Pillar 2, verification half). Chapter 6 presents the bootstrap experiments, covering teacher filtering, self-harvest, and denoising distillation, and ends with the five-system scoreboard above (Pillars 1 and 2, learning half). Chapter 7 analyses the DPO near-miss pathology and the mechanism evidence for the one-sentence thesis (Pillar 3). Chapter 8 concludes with limitations, including the LLM-generated character of the benchmark surfaces and the boundaries of the strict-holdout evidence, and with directions for moving more of the blind region across the projection line.

9 Background and Related Work

Chapter 1 stated the one-sentence thesis and the three-layer design that carries it. This chapter places that design in the literature and marks exactly where it differs from its nearest neighbours. CICADA sits at the junction of four literatures: knowledge-graph question answering (KGQA) benchmarks and semantic parsing; verifier-guided generation and self-correction; bootstrapped self-training and distillation; and LLM-over-KG agents. Its overall shape (generate candidate outputs, filter them, retrain on the survivors) is deliberately *not* claimed as novel. That loop is by now a standard instrument, and several systems surveyed below share it almost exactly. The dividing line this chapter draws against the nearest neighbours runs along two axes only. The first is **filter strength**: what evidence a candidate must present before it is allowed to become supervision. This may be gold-answer agreement, a self-consistency vote, bare execution success, or, in our case, a deterministic multi-stage verifier plus a training-time-only oracle gate. The second is **blind-region handling**: what happens to the candidates that pass the filter *wrongly*. They may silently poison the training pool, or they may be surfaced, labelled, and put to work. The reader should finish this chapter able to predict, for any related system, where its filter is weaker than a deterministic executable check and what that system does with its blind region.

10 KGQA benchmarks and semantic parsing

Question answering over structured knowledge has a long semantic-parsing tradition: map a natural-language question to a formal query, execute it, return the result. The large modern KGQA benchmarks, GrailQA (Gu et al., 2023), LC-QuAD 2.0 (Dubey et al., 2019), and KQA Pro (Cao et al., 2022), established the construction recipe this work adapts. The recipe enumerates logical forms or query templates first, verbalises them into canonical questions, then paraphrases for surface diversity, with crowd workers typically performing the verbalisation and validation stages. GrailQA also established the now-standard evaluation split taxonomy (i.i.d., compositional, zero-shot), which foregrounds whether a parser generalises beyond the query shapes it was trained on.

CICADA’s benchmark (Chapter 4) adopts this template-first recipe but replaces the crowd stages with LLM stages. In particular, it replaces crowd *validation* with mechanical fidelity checks. The substitution is motivated by a measured failure rather than an assumption. An LLM checker asked to validate paraphrase fidelity approved every one of 9,762 rewrites presented to it, including rewrites containing known semantic drifts (project measurement; see results table, promoted rows). An uncertain model validating an uncertain model’s output detects nothing; only deterministic projection checks (entities, literals, numbers, cue words preserved) provide a signal. We turn the same scepticism on the benchmark itself. Benchmark correctness in this literature is usually asserted, whereas Chapter 4 *measures* it. Every answer oracle is implemented twice from independent code paths, and their agreement rate is reported: 99.88% (14,752/14,770) (dual-oracle audit; results table, promoted rows).

One further gap in the benchmark literature matters here. Mainstream KGQA benchmarks contain almost exclusively answerable questions. Unanswerable or ill-posed questions, and therefore abstention as a scored behaviour, are largely absent, a gap noted in work on selective prediction and answerability detection for QA (Rajpurkar et al., 2018; Kamath et al., 2020). For a public-accountability domain this omission is not tolerable. Chapter 4 builds three families of abstention traps directly into the benchmark, and §2.6 tracks abstention support as a column of the positioning table.

11 Verifier-guided generation and self-correction

A second literature asks whether a model’s own outputs can be improved after the fact. The negative result is by now well established: *intrinsic* self-correction, where a model critiques and revises its own reasoning with no external signal, tends to degrade rather than improve reasoning performance (Huang et al., 2024a). The positive results all share one feature: the feedback comes from an environment, and it is *typed*. Self-Debugging repairs code against execution traces and error messages (Chen et al., 2023). CRITIC couples generation to external tools whose outputs ground the critique (Gou et al., 2024). In text-to-SQL, execution-guided decoding (Wang et al., 2018) and the decomposed prompting pipelines DIN-SQL (Pourreza & Rafiei, 2023) and MAC-SQL (Wang et al., 2024) route schema and execution errors back into targeted repair. QueryAgent’s ERASER performs stepwise self-correction from typed environment feedback in KGQA (Huang et al., 2024b). The prescription that emerges is that repair works when the error is diagnosed by something other than the model, and the diagnosis names the error type.

CICADA’s *reflector* follows this typed-error prescription exactly, with one positioning rule that this dissertation maintains strictly: the reflector is a *consumer* of verifier signal, never a verifier itself. The four deterministic checks (Chapter 5) produce diagnostic signals: compilation failures, grounding failures, structural mismatches, and, at harvest time, oracle disagreement. The reflector, an LLM and therefore uncertain, consumes those diagnostics to propose a repaired plan, and the repaired plan re-enters the same four checks. Nothing the reflector says is ever trusted on its own authority. This design decision keeps the architecture out of the "uncertain model verifies uncertain model" regress of §2.1. The reflector also adds one repair behaviour with no direct analogue in the text-to-SQL repair literature: provenance-gated empty-result handling. An empty result whose filter literals all trace verifiably to the question is an *answer* (there genuinely are no such contracts), not a defect to be repaired away. Only an empty result with untraceable literals is a repair trigger. A development-slice measurement motivates the distinction. A naive "empty means broken, so retry" repair loop was net-negative on the ~50-question dev_smoke slice: accuracy dropped from 84% to 66%, and hallucinated answers rose from 1 to 6 (a development-slice measurement, restated with its gate in Chapter 5, §5.5; motivation only, never a comparative claim).

12 Self-training, rejection sampling, and distillation

This is the section in which the dividing line must be drawn, because the loop shape here is genuinely shared.

12.1 Filtered self-training loops

STaR (Zelikman et al., 2022) established the modern pattern: sample rationales from the model, keep those whose final answer matches the gold label, fine-tune on the survivors, iterate. Rejection-sampling fine-tuning generalises the pattern. RFT scales it for mathematical reasoning (Yuan et al., 2023), and RAFT frames it as reward-ranked filtering (Dong et al., 2023). ReST (Gulcehre et al., 2023) and ReST-EM (Singh et al., 2024) cast the grow-filter-train cycle as expectation-maximisation, and show that a model can improve on problems whose solutions it filtered itself. All of these accept a model output into the training pool on a *pass criterion*. In all of them the pass criterion is the filter’s entire theory of correctness: a gold final answer, a reward-model score, or a binary success signal. None of them examines closely the population of outputs that satisfy the criterion for the wrong reasons. The right answer reached by an invalid rationale in STaR is the canonical example. In all of them that population enters the training pool indistinguishably from the genuinely correct one.

12.2 Nearest neighbours: ExeSQL and SCD

Two recent systems are close enough to CICADA that the boundary must be drawn with care. Both deserve to be treated as serious neighbours rather than foils.

ExeSQL [ExeSQL; EMNLP 2025 Findings] bootstraps text-to-SQL models across SQL dialects using execution-guided rejection sampling. Candidate queries are filtered by whether they execute, the survivors form supervised data, and iterative DPO uses execution-failing queries as preference negatives. Of the pub-

lished work we are aware of, this is the loop most similar in shape to Chapter 6: an executable environment supplies the filter, and the filtered pool drives both supervised and preference-based training of a local model. **SCD** [SCD; arXiv:2511.07998] is a two-stage self-consistency distillation pipeline for structured-data question answering. A teacher-distillation stage is followed by a self-distillation stage, with self-consistency voting supplying the filter. It shows, as we do, that 8B-scale students can reach strong structured-QA performance without gold logical forms.

The boundary against both is exactly the two axes announced above. First, filter strength. ExeSQL’s filter is execution success; SCD’s is agreement among samples. Both are correctness *proxies*: a SQL query can execute cleanly and compute the wrong thing, and a model can be consistently wrong. CICADA’s filter is a deterministic compile–ground–execute–verify chain with closed slot enumerations at every projectable point, plus an oracle-agreement gate at training time. The oracle gates but never authors: it discards candidates, and contributes no tokens to any training example. We do not claim that this makes the filter complete; we claim, and measure, precisely where it is incomplete. Second, blind-region handling. Because prior filters treat their pass criterion as truth, the passing-but-wrong population is invisible to them by construction. We characterise that population directly. During self-harvest on bridge-type questions, 86.7% of generated plans passed the deterministic verifier while only 52.3% agreed with the oracle, a 34.4-point verifier-blind stratum on this question type (self-harvest audit, n=1,351; results table, promoted rows). That stratum is exactly the failure mode execution-success filtering cannot see, and it exists at scale.

12.3 The divide-line

The contribution of this dissertation relative to this literature is therefore *not* the generate–filter–retrain loop. It is two things. The first is the filter: a deterministic, executable, multi-stage compile–ground–execute–verify chain with closed slot enumerations, plus a training-time oracle that gates and never authors. The second is the blind-region handling: verifier-passing-but-wrong outputs are not left to poison the pool silently. They are routed into *labelled hard negatives* for preference learning, while correct abstentions become abstention supervision rather than being discarded as non-answers. On this foundation, Chapter 6 shows that a precise-but-partial-coverage filter suffices to bootstrap *both* stages of a neuro-symbolic stack, the understanding stage and the planning stage, past the cloud teachers that initialised them. Chapters 4–5 show the residual blind region being closed by abstention rather than ignored. Where ExeSQL asks "does it run?" and SCD asks "does it agree with itself?", this dissertation asks which properties can be checked deterministically, what leaks past those checks, and what the leak should be used for.

12.4 Preference optimisation and its pathologies

One consequence of harvesting hard negatives is that preference optimisation becomes available. Chapter 7 reports that using near-miss hard negatives in DPO (Rafailov et al., 2023) can be actively harmful. The pathology has recent theoretical grounding. Razin et al. [2024] show that DPO can produce *likelihood displacement*, in which the probability of the preferred response falls alongside the dispreferred one. The effect is most severe when the pair is similar, which is precisely the regime of verifier-passing near-misses that differ from a correct plan by one slot or one relation. One proposed remedy is DPOP, which adds a penalty preventing the positive’s likelihood from falling (Pal et al., 2024). Another is IPO, which replaces DPO’s unbounded objective with a bounded one that resists overfitting to the preference margin (Azar et al., 2024). This dissertation engages with this literature empirically rather than theoretically. Chapter 7 uses it as the interpretive frame for a mechanism-explained negative result contrasting subtractive preference suppression with additive distillation, and evaluates an IPO variant among the mitigations.

13 LLM-over-KG agents

A parallel line couples LLMs to knowledge graphs at inference time. Pangu (Gu et al., 2023) keeps generation on a leash: the LLM *discriminates* among candidate logical-form extensions enumerated from the graph, rather than generating freely. This guarantees groundedness by construction, at the cost of a search procedure in the loop. ChatKBQA (Luo et al., 2024a) has a fine-tuned LLM generate a logical form and then grounds

Table 2: Positioning against the nearest neighbours: filter strength, blind-region handling, stages distilled, and abstention support.

System	Filter type
STaR (Zelikman et al., 2022)	gold final-answer match
RFT / RAFT (Yuan et al., 2023)	answer match / reward-model score
ReST / ReST-EM (Gulcehre et al., 2023; Singh et al., 2024)	scalar reward threshold
ExeSQL [EMNLP 2025 Findings]	execution success (+ iterative DPO on execution failures)
SCD [arXiv:2511.07998]	self-consistency vote (after teacher stage)
Pangu (Gu et al., 2023)	grounded enumeration (inference-time)
ChatKBQA (Luo et al., 2024a) / RoG (Luo et al., 2024b)	retrieval grounding of generated forms/paths
CICADA	deterministic compile-ground-execute-verify, closed slot enums; train

its entities and relations by retrieval, decoupling generation from grounding. RoG (Luo et al., 2024b) plans relation paths that are grounded against the graph and then reasons over the retrieved paths. StructGPT (Jiang et al., 2023) gives a frozen LLM iterative read access to structured data through specialised interfaces.

CICADA shares this family’s premise, that the graph and not the model is the ground, but differs in where authority sits and in what failure produces. First, the KG executor is the *sole answer authority*. The LLM never composes the answer text from retrieved material; it produces a plan, and only the deterministic execution of that plan produces an answer. Second, the plan is a *checkable artifact*: a typed graph plan compiled by a deterministic twelve-transform compiler, so conformance to the executable contract is a mechanically decidable property rather than a hope about generation. Third, grounding failures are *diagnosable events* rather than dead ends. A failed grounding emits a typed diagnostic that feeds the gated repair of §2.2 or, when repair is exhausted, a principled abstention. Where Pangu prevents ungrounded output by restricting the model to discrimination, CICADA permits generation and then subjects it to checks whose failure modes are themselves informative. This design difference is what makes the harvest of §2.3 possible in the first place.

14 LLM-generated evaluation data

Using LLMs to generate benchmarks invites a known pathology: naive paraphrasing collapses toward a house style, which overstates surface diversity while narrowing it [see e.g. Yu et al. 2023, AttrPrompt]. Chapter 4’s generation pipeline conditions paraphrasing on six explicit style axes with mechanical acceptance checks. In keeping with this chapter’s theme of measuring what is usually asserted, it reports distributional evidence rather than a diversity claim: the median pairwise trigram-Jaccard similarity among accepted surfaces is 0.429 (distributional measurement; results table, promoted rows). Two disciplinary notes bound the claim. First, we do not assert that these LLM-generated surfaces are more "natural" than crowd-written ones. The opposite concern, that they are less natural, is a documented limitation taken up in Chapter 8; the claim is measured correctness and measured diversity, nothing more. Second, paraphrase must preserve *abstention cues*. A rewrite that smooths away the ambiguity that made a question unanswerable silently converts an abstention item into an answerable one. Cue preservation is therefore enforced as a benchmark-integrity device (Chapter 4).

15 Positioning summary

Table 2.1 operationalises the divide-line. Each neighbour is scored on the two axes of this chapter, filter strength and blind-region handling, together with abstention support. No column concerns loop shape, because loop shape is not where the difference lies. No accuracy numbers appear, because numbers measured on different benchmarks are not comparable, and none of this dissertation’s numbers were measured on theirs.

Table 2.1 — Positioning against neighbour systems.

Two boundary clarifications keep the table honest. The oracle-agreement gate exists **only during training-data harvest**. At evaluation time no oracle signal enters the pipeline, since that would constitute answer leakage, so the inference-time system is filtered exactly as strongly as its deterministic checks, no more. And the reflector does not appear in the filter column anywhere: it consumes the filter’s diagnostics, and its outputs re-enter the same checks, so it adds repair capacity, not filter strength.

With the neighbours placed, the remaining chapters proceed constructively. Chapter 3 describes the data, and Chapter 4 presents the benchmark whose integrity apparatus §2.1 motivated. Chapter 5 presents the three-layer system whose checks define the verifiable region. Chapter 6 presents the bootstrap experiments that draw supervision from it, and Chapter 7 analyses what leaks past it.

16 Data Engineering and Knowledge-Graph Construction

Hard verification is the organising principle of this dissertation, and it is only possible over data whose semantics are explicit. A verifier cannot check that a sum is legitimate unless the data records which values may be summed. An oracle audit cannot re-derive an answer unless every convention behind the original computation is written down as data rather than remembered informally. This chapter describes how we turned five years of UK Open Contracting Data Standard (OCDS) releases into such a substrate: a typed, convention-explicit, audit-first knowledge graph of 215,221 contract-award nodes and 131,502 canonical organisations [ARTIFACT-VERIFIED: data/kg/nodes/contract_nodes.parquet, data/kg/nodes/org_nodes.parquet]. Two design stances run through the chapter. The first is *precision-first entity resolution*. A false merge corrupts every downstream count and sum, so merging is tiered, conservative, and audited. The documented cost is under-merging, not silent over-merging. The second is *conventions as data*. The additivity of money values, the flat first-party record universe, and the latest-release snapshot are checkable flags and stated choices, not implicit assumptions. Chapter 4’s independent oracle audit and Chapter 5’s verifier both depend on exactly these properties.

17 Source data and ingestion

17.1 The OCDS release corpus

The source data is the UK’s public OCDS publication covering Contracts Finder and Find a Tender notices. It is distributed as one compressed JSON-Lines file per year for 2022–2026. Each line is a *compiled release*: a self-contained JSON snapshot of one contracting process, identified by a stable **ocid**. A release carries the buyer reference and a **parties** array with the full organisation records. It also carries a **tender** block (title, CPV classification, estimated value, procurement method, lots, periods), an **awards** array (suppliers, award values, related lots), and a **contracts** array (signed values and dates). The five year-files contain 166,277 release records in total (source analysis: ocds_data_analysis.md). Volume jumps sharply in 2025, when the Procurement Act 2023 introduced new mandatory notice types. A release is a snapshot of a *process*, not a contract: several releases of the same OCID can appear within and across year files as the process moves from notice to award to signature.

Ingestion (`src/procurement_graph/ingest/loader.py`) streams each gzipped file line by line, so no file is ever loaded whole. Each release is parsed defensively: malformed lines are logged and skipped, and rows missing **ocid** or **date** are dropped. Each release is then flattened into one row, with the nested **parties** and **contracts** arrays serialised as JSON strings, so the interim Parquet file has a simple and universally readable schema. Deduplication keeps, for every OCID, the release with the latest ISO-8601 **date**; string comparison is safe because all dates are ISO-formatted. A **source_years** column records every year-file in which the OCID was observed.

The *latest-release snapshot* is the first explicit convention of this chapter, and it is a choice with alternatives. Modelling full amendment chains would preserve every correction, but it multiplies schema complexity for question families the benchmark never asks. Keeping the first release would discard corrections. Taking the latest release matches the publisher’s own compiled presentation and gives exactly one authoritative snapshot per contracting process. The convention is stated here and stress-tested by the oracle audit in Chapter 4.

The knowledge graph therefore answers questions about the latest published state of each process within the ingested corpus. It does not answer questions about publication history, and it does not cover UK procurement in its entirety.

A second, wider extraction pass (`src/procurement_graph/extract/tables.py`) re-reads the raw files to capture fields the first pass does not. It produces seven tables (tender core, lots, award criteria, awards, bid statistics, text evidence, documents). Alignment with the deduplicated snapshot is enforced structurally, not by convention: a raw release is accepted only when both its `ocid` and its release `id` match the winning release chosen at ingestion. This guarantees that the extracted tables are row-for-row consistent with the interim snapshot.

17.2 Quality problems the pipeline must absorb

Three families of data-quality problems shaped the design. All were documented at analysis time, and all are visible in the code paths that handle them.

Money semantics. Where a contract's value lives is inconsistent across the corpus. In the 2024 population, `awards[].value` was null in every inspected record. The reliable signed value sits in `contracts[].value.amount`, joined to its award through `awardID`. `tender.value.amount` is a pre-award *estimate*, present in roughly a third of records (source analysis: `ocds_data_analysis.md`). Worse, the corpus mixes genuinely additive spend with framework-agreement ceilings — multi-year umbrella values that reach tens of billions of pounds — so a naive SUM over "contract values" is not total spend. The extraction layer therefore records a `value_source` per award with a fixed precedence (`award`, then `contract`, then `tender`) and never overwrites provenance. The graph derives a boolean `value_is_additive` that is true exactly when the value came from an award or a signed contract, and false when it is a tender estimate. In the final graph, 160,362 award rows carry contract-sourced values, 15,640 award-sourced, 19,623 tender-sourced, and 19,596 no value at all; 176,002 of 215,221 rows are flagged additive [ARTIFACT-VERIFIED: `data/kg/nodes/contract_nodes.parquet`]. This is the "conventions as data" stance in miniature. Instead of the pipeline privately knowing that tender estimates must not be summed, the flag turns *additive-only money aggregation* into a guard that the downstream executor and verifier enforce mechanically (Chapter 5). The build itself also validates the flag: the KG validation suite fails the build if any award- or contract-sourced value is marked non-additive, or any tender fallback is marked additive (`src/procurement_graph/kg/validate.py`).

Organisation identity noise. The record-level `buyer` field is a bare `id`, `name` reference. The authoritative organisation records live in `parties`, and the identifiers they carry are of very unequal quality. Around nine in ten buyer references and 86% of supplier references use GB-FTS platform identifiers. These are per-profile account IDs on the publishing platform, not canonical entity identifiers: the Ministry of Defence appeared under 77 distinct GB-FTS IDs in a 2024 sample (source analysis: `ocds_data_analysis.md`). Name strings are noisy even when official identifiers exist (casing variants, ampersand versus "and", trailing whitespace, legal-name drift after mergers). Some source strings are simply malformed. Organisation names with duplicated suffixes, such as "... NHS Trust Trust Headquarters", appear verbatim in the data and later surface in Chapter 7's error analysis. On such an unresolvable mention, the correct system behaviour is a provenance-gated abstention rather than a guessed match [WORKLOG-SOURCED: `cicada_worklog.md`, 2026-07-05]. Section 3.3 describes the entity-resolution design these problems dictate.

Structural multiplicity. One release may hold several awards. One award may name several suppliers (`awards[].suppliers` is a list). Lots relate awards to descriptions many-to-many. The pipeline never collapses these silently: awards are extracted one row per (`ocid`, `award_id`) with the supplier list serialised intact, and the graph gives multi-supplier awards one edge per supplier organisation (Section 3.2).

18 Graph schema and coverage

The knowledge graph (built by `src/procurement_graph/kg/`, orchestrated from `pipelines/40_build_kg.py`) is a set of typed Parquet tables — four node tables and four edge tables — whose column contracts are frozen in code (`kg/schema.py`).

Table 3: Entity-resolution tiers: alias counts merged at each confidence tier, with the residual singletons.

Table	Rows	Key
<code>contract_nodes</code>	215,221	<code>contract_node_id = contract:ocid:award_id</code>
<code>org_nodes</code>	131,502	<code>canonical_id</code>
<code>cpv_nodes</code>	3,870	<code>cpv_code</code>
<code>evidence_nodes</code>	535,731	<code>evidence_id</code>
<code>buyer_of</code>	215,218	one edge per contract node with a resolvable buyer
<code>supplier_of</code>	334,063	one edge per (supplier organisation, contract node)
<code>categorized_by</code>	164,691	contract $\rightarrow$ CPV code
<code>evidence_for</code>	1,326,240	evidence $\rightarrow$ contract, with match scope

All row counts verified directly [ARTIFACT-VERIFIED: `data/kg/nodes/\*.parquet`, `data/kg/edges/\*.parquet`].

The unit of analysis is the *contract-award*: one node per (`ocid`, `award_id`) pair, spanning 118,088 distinct OCIDs [ARTIFACT-VERIFIED: `data/kg/nodes/contract_nodes.parquet`]. Each contract node carries three groups of fields. The award’s own fields cover status, title, dates, and value with `value_source` and `value_is_additive`. The release-level tender context, joined in by OCID, covers title, CPV code and description, procurement method and category, and periods. Derived conveniences are computed once at build time rather than at query time: `has_award_signed_date`, `has_contract_period`, day-count intervals, and contract duration. Organisation nodes carry the canonical name and identifiers, the entity-resolution status and alias counts, region evidence aggregated across all observations of the entity, and precomputed role summaries. The role summaries include buyer/supplier contract counts, first and last activity years, and additive-only value sums, in which non-additive values contribute zero by construction (`kg/enrichment.py`). CPV nodes carry the code, its description, its division/group/class prefixes (which make the CPV hierarchy navigable by prefix), and the same additive-only aggregates.

Coverage is reported as measured fact, not assumption. 215,218 of 215,221 contract nodes have a buyer edge. 204,186 have at least one supplier edge (334,063 supplier edges in total; 22,075 awards name more than one supplier organisation, and one names 249). 164,691 carry a CPV classification, and 215,202 have at least one text-evidence pointer [ARTIFACT-VERIFIED: `data/kg/edges/\*.parquet`]. Among organisations, 7,125 appear as buyers, 114,281 as suppliers, and 1,269 in both roles [ARTIFACT-VERIFIED: `data/kg/nodes/org_nodes.parquet`]. These numbers matter downstream because they fix abstention semantics. A question about the supplier of an award with no supplier edge must produce an empty result and an abstention, not an invented supplier. The benchmark’s empty-result traps (Chapter 4) are sampled with these bounds in mind.

Why this shape? The workload is aggregate and multi-hop question answering: counts, sums, rankings, extremes, and bridge joins such as "contracts whose supplier is also a buyer" or "top suppliers within a CPV category for a given buyer". Two properties of the schema serve exactly this workload. First, every queryable property is a *typed slot with a closed value space*. Procurement category is one of goods, services, or works; CPV codes come from the 3,870-row node table; organisation references resolve against the canonical table. This gives the planner in Chapter 5 concrete grounding targets, and it gives the verifier closed enums to check against. Second, the graph flattens deterministically into a single record universe. The query backend (`src/procurement_graph/qa/benchmark/kg_interface.py`) joins the edges into one row per contract node. Each row carries the full tuples of buyer and supplier canonical IDs, names, and regions, together with scalar `buyer_name/supplier_name` columns defined as the first element of the sorted name tuple. This *flat first-party record universe* is the second stated convention. Filters and aggregations are defined over one row per contract-award with a deterministic first-party scalar. Multi-party membership remains queryable through the tuple columns and `supplier_count`, and the independent oracle of Chapter 4 recomputes answers under exactly the same stated flattening.

A deliberate non-decision completes the schema story: the graph is columnar Parquet, not a graph database. At this scale — a few hundred thousand nodes and about two million edges — vectorised dataframe joins

comfortably outperform query-engine round-trips. The executor stays deterministic, dependency-free, and unit-testable, and the independent oracle audit can re-derive every answer with plain dataframes and no shared query engine. This is an argument about operational surface, not expressiveness: nothing in the workload requires traversals that columnar joins cannot express.

19 Entity resolution: conservative, tiered, audited

The alias universe contains 204,711 raw organisation identifiers, 76.9% of them notice-scoped GB-FTS platform IDs, resolving to 131,502 canonical organisations [ARTIFACT-VERIFIED: data/entities/alias_map.parquet, data/entities/canonical_orgs.parquet]. A single false merge silently corrupts every count and sum that touches the merged entity. Resolution is therefore organised as ordered tiers of strictly decreasing evidence strength, each recorded per entity in an `er_status` field, and everything below the evidence bar is left unmerged. The realised tier sizes are: registry-identifier deterministic merges 26,704; safe deterministic variants 6,332; normalised name + region 14,875; name-only 1,286; LLM-adjudicated borderline cases 255; curated government lookup 37; and 82,013 singletons deliberately left unmerged [ARTIFACT-VERIFIED: data/entities/canonical_orgs.parquet, `er_status` value counts].

Phase 1 — deterministic, registry-anchored (`src/procurement_graph/er/phase1.py`). Any party observed with an official registry scheme takes that registry identifier as its canonical ID. The schemes are Companies House (GB-COH), NHS organisation codes (GB-NHS), UK Provider Reference Numbers (GB-UKPRN), the charity registers (GB-CHC, GB-SC, GB-NIC), and the Mutuels register (GB-MPR). A fixed priority order applies when several schemes are observed for one raw ID. A single, tightly scoped cross-referencing rule then attaches platform IDs to these anchors: when, *within one OCID*, a GB-FTS party and an officially identified party share the same normalised name, the GB-FTS ID becomes an alias of the official entity. The OCID scoping is the safety property. Co-occurrence of both identifiers on the same notice is direct evidence that they denote one organisation; the same name match across unrelated notices would not be. GB-FTS identifiers are never themselves canonical. They are notice-scoped platform artefacts, and treating them as entity keys causes exactly the fragmentation failure (one ministry, 77 nodes) the design exists to avoid.

Phase 2 — heuristic but bounded (`src/procurement_graph/er/phase2.py`). Entities left unresolved by Phase 1 pass through three steps. First comes a small curated lookup of well-known government bodies (37 canonical entities, e.g. GOV-MOD for the Ministry of Defence, which ultimately absorbs 1,222 aliases, the largest alias group in the table [ARTIFACT-VERIFIED: data/entities/alias_map.parquet]). Second comes exact normalised-name + region grouping. The normaliser (`common/normalise.py`) uppercases, strips accents and punctuation, rewrites ampersands, and removes legal-form suffixes (LIMITED, LTD, PLC, LLP, CIC, ...); groups of more than one raw ID sharing a normalised name and region merge under a stable content-hashed `MERGED-*` identifier. Third, name-only merging is the fallback for entities with no region evidence. Everything still unmatched becomes a *singleton* and keeps its own raw ID as a provisional canonical ID. Fuzzy string matching was deliberately demoted: the Jaro-Winkler candidate generator (`er/candidates.py`) is a read-only analysis tool with blocking and hard output caps. It produces human-review files and *never* writes to the entity tables.

Safe-merge extension — audited, cluster-checked. A later ablation framework (`src/procurement_graph/experiments/er_ablation.py`, applied by `scripts/apply_safe_er_merges.py`) generated candidate pairs with risk flags (scheme conflicts, parent/child and subsidiary patterns, acronym ambiguity, procurement-agent patterns). The deterministic-safe subset and an LLM-adjudicated borderline queue then passed through cluster-level checks, and the surviving merge edges were applied by union-find. Canonical-ID election within a merged component prefers official registry IDs (in scheme-priority order), then curated GOV-* IDs, then existing `MERGED-*` IDs. The LLM tier contributes only 255 of 131,502 canonical entities, and every one carries `er_status='llm_safe_merge'` so it can be inspected, discounted, or reverted wholesale. The two extension tiers account for the `deterministic_safe_merge` (6,332) and `llm_safe_merge` (255) statuses in the audit trail.

The audit trail. Every canonical entity — merged or singleton — has one row in `er_audit.csv`. The row records its canonical ID, display name, resolution status, official scheme, alias count, and the full pipe-

delimited list of raw IDs it absorbs (131,502 rows [ARTIFACT-VERIFIED: data/entities/er_audit.csv]). Together with the `alias_source` column of the alias map (phase1 / phase2 / safe-merge / unresolved), every merge decision in the graph can be reconstructed after the fact. The build also verifies referential closure: every alias target must exist as an organisation node, or the build fails.

Residual ambiguity is handled at query time, conservatively. The design makes no recall claim. Its documented cost is that decoration variants of one organisation — "The X" versus "X", "X (BEIS)" versus "X", casing differences — can survive as distinct canonical rows, splitting one organisation's records across variants. The runtime policy (`src/procurement_graph/reasoning/entity_resolution.py`) is the single shared gate through which every planner and repair path resolves an organisation mention, so no caller silently takes the top hit of a weak match. Its logic, directly from the code, is as follows. An exact record match or a ≥ 0.999 score resolves immediately. Otherwise the top candidate must score at least `min_score = 0.85`, and if a second candidate also clears that bar, the top must lead by a margin of at least `min_margin = 0.10`. There is one exception. When all contenders collapse to one *decoration-variant equivalence key* (casefold; strip a leading "the" and a trailing parenthetical alias; unify "&"/"and"; drop punctuation), the mention is resolved, but *every* variant is surfaced. The executor then filters with an IN over all of them rather than undercounting on one. Genuine ambiguity, meaning distinct equivalence keys within the margin, refuses to resolve and feeds the abstention path. A minimal, evidence-driven mention-expansion table is applied only when the literal lookup finds nothing. It currently holds the single rule ICB $\rightarrow$ "Integrated Care Board", added when the audit showed the graph stores full names while questions use the abbreviation. Both mechanisms were introduced *after* the Chapter 4 oracle audit identified variant splits and abbreviation mismatch as the ambiguity classes with practical impact. This follows the thesis-wide discipline: conventions and repairs are adopted in response to measured failures, then written down.

20 Provenance: evidence nodes and why they exist

The fourth node type stores no structured facts at all. The 535,731 evidence nodes hold the corpus's long free-text fields — lot descriptions (332,802), tender descriptions (164,373), selection-criteria descriptions (21,429), review details, and performance terms [ARTIFACT-VERIFIED: data/kg/nodes/evidence_nodes.parquet]. They are extracted verbatim with their OCDS field paths and cover 166,257 distinct OCIDs, essentially the whole release corpus. They are deliberately kept out of the structured record universe. Paragraphs are not queryable slots, and storing them as contract-node attributes would blur the line between what the executor can filter on and what it can only cite.

Their purpose is provenance. The `evidence_for` edge table (1,326,240 edges reaching 215,202 of 215,221 contract nodes) links each text fragment to the contract-award nodes it describes. The link is at lot scope when the fragment's lot ID matches a specific award's related lots, and at OCID scope otherwise. The resolution is recorded in a `match_scope` column (`lot`, `ocid`, or `ocid_fallback`), so the precision of every provenance link is itself queryable. Downstream, this is what makes *provenance-gated abstention* concrete rather than rhetorical. Every answer must trace to specific records, and answers are returned with evidence samples attached. When a question's constraints match no records, such as an unresolvable "NHS Trust Trust" mention or a year with no awards for the named buyer, the system returns an explicit empty-evidence abstention instead of a fluent guess. The benchmark's empty-result abstention family (Chapter 4) tests precisely this contract.

21 Engineering properties that matter scientifically

Several build-level decisions are reported here because downstream scientific claims rest on them, not because they are implementation trivia.

Determinism. The build is a pure function of its inputs: no randomness, stable sort orders before every deduplication, content-hashed merge identifiers, and modal-value tie-breaks resolved by deterministic ordering. Phase 2 of entity resolution is explicitly idempotent: re-running it on already-processed outputs repairs rather than duplicates canonical rows (`consolidate_duplicate_canonical_orgs`). Deterministic rebuilds make the Chapter 4 oracle audit meaningful, because two independent implementations can only

be compared over a reproducible substrate. They also keep the benchmark’s gold programs stable across the project’s lifetime.

Validation-gated writes. The builder runs a validation suite: primary-key uniqueness for every node table, (`ocid`, `award_id`) uniqueness, referential closure for every edge endpoint, alias-target closure, value-additivity consistency, and coverage measurements. It *refuses to write* the graph if any check fails, and coverage numbers are emitted as warnings into a machine-readable report (`kg/build.py`, `kg/validate.py`). The counts quoted in Section 3.2 are therefore properties the artifact is guaranteed to have, not properties it happened to have once.

Stable composite keys. `contract_node_id = contract:ocid:award_id` names the unit of analysis, and every edge ID embeds its endpoints (`buyer:canonical_id:node_id`). Edges are therefore self-describing and can be deduplicated with set semantics. When one award names several raw supplier IDs that resolve to the same canonical organisation, the build collapses them into a single edge whose `raw_id` field carries the sorted JSON list of contributing raw IDs. Multiplicity is normalised away from the graph’s semantics but preserved in its provenance.

Empty strings are a first-class hazard. Missing values throughout the corpus arrive as empty strings, not nulls, and the build treats them accordingly. Supplier IDs that are empty or unresolvable produce no edge rather than an edge to an empty organisation. CPV edges are emitted only for non-empty codes. Derived flags (`has_award_signed_date`, `has_contract_period`) are computed by explicit non-empty tests. This hazard escaped into computation once. In an early bridge-family oracle, an empty-string party name matched every record lacking that party, inflating every bridge sum by a constant. The dual-oracle agreement audit caught it, and Chapter 4 dissects it. It is the concrete illustration of why this chapter insists that conventions be checkable.

22 What the knowledge graph hands to the rest of the thesis

The chapters that follow consume the substrate built here through a deliberately small contract. The *grounding targets* are the typed slots and closed value spaces of Section 3.2: the planner must land on real fields, real CPV codes, and canonical organisations, and anything else is mechanically rejectable. The *verifier guards* are the explicit conventions: additive-only money aggregation via `value_is_additive`, the flat first-party record universe, the latest-release snapshot, and the decoration-variant IN-filter policy. The *abstention semantics* are fixed by the measured coverage bounds and by provenance: empty result sets are meaningful, expected outcomes with dedicated benchmark traps, not failure modes. None of this makes the graph "ground truth". Correctness downstream is always relative to the conventions this chapter documents, and Chapter 8 lists this relativity among the limitations. Nor does the graph claim complete coverage of UK procurement; it covers the ingested corpus only. What it does claim, supported by the artifact-verified validation gates, is that the residual ambiguity of the substrate is documented and bounded. That is the property everything downstream is built against. Chapter 4 constructs a 12,828-question benchmark by executing parameterised plans directly against this graph, and Chapter 5 builds the compile-ground-execute-verify chain over the same typed interface.

23 Benchmark Construction and Split Discipline

Chapter 3 built the knowledge graph; this chapter builds the instrument that measures systems over it. Every empirical claim in this dissertation is only as credible as the benchmark beneath it. We therefore treat the benchmark not as a fixed asset but as an *instrument with measured error*. It contains 12,828 questions, each generated from a fully parameterised gold program. Its oracles are validated by an independent second implementation at 99.88% agreement. Its three abstention classes are designed as traps rather than decoration, and its plan-level split isolation is verified at zero train/test plan overlap. An honesty protocol also governs how the benchmark is *used*. The 260-question comparison set is a development set, because the pipeline-v2 fixes were derived from its errors. The 2,285-question final test set is confirmatory, and it is dual-reported with and without the nineteen rows that string-match development-derived cue lists. Every adaptive-overfitting risk identified in review is disclosed in this chapter, not patched over.

24 Plan-first generation: L1 templates and L2 persona rewriting

The construction principle throughout is *plan-first*: a question is never written and then annotated. Instead, a parameterised logical plan is instantiated against the knowledge graph, its oracle answer is computed by execution, and only then is a natural-language surface generated. Post-hoc annotation of collected questions would be more naturalistic, but its coverage cannot be verified and its cost lies outside a final-year project. Plan-first generation guarantees the property this project needs most: every question carries an executable gold program, which is exactly the training target of Chapters 5–6.

Level 1 (L1). Thirty-three template families instantiate fully parameterised plans against the graph, execute them to obtain oracles, and verbalise canonical questions (9,044 generated; 6,772 retained after a cleaning pass removed hidden presence filters and one ill-posed conjunction family). A second, targeted generation wave (`scripts/build_targeted_v2.py`) added five deliberately harder subsets, each with control-plane validation on every run. *naturalized* is an LLM rewrite of count/sum/factoid questions, accepted only after a faithfulness gate and knowledge-graph revalidation of the oracle, with template fallback on rejection. *coverage_fixed* removes answer-changing guard constraints and recomputes the oracles. *unanswerable* holds the three abstention families of §4.5, each instance schema- or graph-verified. *extended_ops* adds argmax, top-k, distinct-set and two-sided comparison, each with recomputable evidence. *bridge_join* holds pure two-hop semijoins that require decomposition. Factoid questions are anchored on a (buyer, supplier, year, CPV) tuple and admitted only if that anchor resolves to a unique answer value. Count questions are readmitted only if re-execution against the graph reproduces the inherited oracle.

Gold programs as flat specifications. Each row’s `gold_plan` is a flat, executable specification: a list of typed constraints (`field`, `op`, `value`), an answer operation, an answer field, and machine-readable metadata. Two structural devices matter for what follows. First, multi-hop *bridge* questions are expressed by an `in_subquery` constraint whose value is a resolver payload (for example `"resolve": "suppliers_of_buyer", "buyer": ...`). Six resolver types cover the bridge inventory, so a bridge is a first-class, recomputable object rather than a prose description. Second, operations whose parameters originally lived only in the question surface were later backfilled into metadata (§4.2): grouped rankings carry `group_by`, `k` and `metric`, and every comparison carries `compare_params`. Across the whole corpus, comparison questions realise exactly four side types. These are paired entity counts (two independent sub-counts and a comparator), a date axis (direction plus pivot date against a signed-award date), field equality (whether a categorical field equals a named value), and a numeric threshold (whether an aggregate exceeds a stated amount).

Level 2 (L2). An LLM rewrites each surface under four personas: an ordinary citizen, a policy analyst, a public-spending auditor, and an investigative journalist. A *deterministic* per-surface accept/reject gate checks atom preservation, no invented numbers, no invented organisations, retention of unanswerability triggers, and evidence of genuine rewriting. The gate accepted 9,762 surfaces and rejected 1,170 with machine-readable reasons, dominated by invented temporal relations (890). The deterministic gate exists because the obvious alternative fails in practice: an LLM checker asked to validate the same rewrites approved 9,762 of 9,762, including known semantic drifts. All surfaces of a plan share one oracle, so L1-versus-L2 accuracy differences isolate language-to-plan generalisation from everything else.

Surface diversification (v4). Naive LLM paraphrasing collapses toward near-copies, so a final diversification pass conditions rewrites on six explicit style axes: terse search query, verbose two-sentence context, politely embedded request, syntactic flip, multi-sentence background, and typo noise (the last confined to training data). The axes cycle deterministically. Acceptance is again mechanical: every gold literal must appear verbatim, no invented years or codes, a position-aware category-drift rule, and, for abstention questions, mandatory preservation of the cue words (§4.5). In the frozen benchmark, 43.8% of evaluation-split surfaces were rewritten in place with originals retained in metadata, and 2,589 styled variants were added to training. The distributional evidence is a trigram-Jaccard median of 0.429 between rewrite and source. We report this distributional evidence and claim nothing more: the surfaces are LLM-generated under controlled style axes, and we do not claim they reproduce real-user language.

Compositional coverage census. To characterise what the corpus contains structurally, every gold plan was reduced to a structural signature: (answer operation, bridged := any `in_subquery` constraint, group-by axis, comparison side type, filter-slot combination), reconstructed programmatically from the flat specifications. Characterising by template *names* instead would make any novelty assertion circular. Over the census scope of 12,779 rows (12,828 minus the 49-row smoke slice; the four analysis splits), the frozen corpus realises 39 distinct (template family, answer operation) pairs. The five-field signature refines these to **38 fine-grained structural signatures collapsing to 12 coarse cells** (operation $\times$ bridged $\times$ group-by). An earlier ad-hoc count of 50 is reproduced exactly under a finer seven-field signature; it is reported as a granularity variant, not corrected away. The census landmarks are informative in themselves: bridging occurs only under count and sum; grouping occurs only in the non-bridged ranked-top-k family; and comparisons realise exactly the four side types above. The benchmark is therefore broad at the surface level but occupies a characterisable, finite region of compositional space. Chapter 7 uses this fact when it probes held-out compositions, and it is an honest bound on what "coverage" means here.

25 Quality audit v1 $\rightarrow$ v2: six measured defects, six mechanical fixes

An audit of the first-generation benchmark against professional benchmark-construction practice found six defects. Each was *measured* before being fixed, each fix is mechanical, and each fix is itself testable:

1. **Boolean answers were 100% "True"** (330/330), so guessing scored perfectly. Fix: 155 mutated "False" twins (the same surface with the year moved to a zero-record year), each oracle recomputed independently; blind "True" guessing falls to 68%.
2. **Factoid answers drawn from tiny value domains** (42% of factoids) rewarded distributional guessing. Fix: 1,183 rows re-bucketed into a separate `categorical` bucket, leaving the factoid bucket with high-entropy answers.
3. **Unsupported questions were lexically detectable** (52% by a word-list probe). Fix: answerable contrast twins that swap the unsupported attribute for a supported one under otherwise identical wording, so lexical shortcuts no longer separate the classes.
4. **L2 rewrite drift was invisible to the LLM checker.** Fix: a position-aware mechanical drift detector. A category phrase appearing before the bridge relative clause modifies the outer counted noun and cannot be excused by the subquery; after it, with the subquery carrying that category, it is legitimate. Final precision on the corpus: the one true defect isolated, zero false positives.
5. **Gold programs were under-specified:** thresholds, comparison sides and ranking parameters lived only in the surface text. Fix: parameter backfill — 991 additive-money guards, 690 `compare_params` blocks, 316 answer fields, 15 top-k parameter sets — making every plan executable without reading the question.
6. **Oracle circularity:** generator and oracle shared one implementation. Fix: the independent second implementation of §4.4.

26 Splits, balance, and isolation hygiene

Frozen arithmetic. The frozen v4 benchmark contains **12,828 rows = train 9,267 + dev_tune 556 + dev_select 671 + dev_smoke 49 + final_test 2,285** ($9,267 + 556 + 671 + 49 + 2,285 = 12,828$), plus a 7,483-row surplus pool held outside all splits. The 260-question comparison set `compare_set_v4` — 13 evaluation buckets $\times$ 20 questions — is drawn entirely from within `final_test`; its governance is the subject of §4.6.

How rows are assigned. Splitting operates on *groups*, not rows (`scripts/merge_stratify_l1_l2.py`). Every row receives a near-duplicate group key that hashes its template family, question type, answer operation, expected status, and the normalised entity/year/CPV/category content of its constraints. All surfaces

of a plan, and all near-duplicate parameterisations, share a group, and whole groups move together, so no paraphrase of a test question can sit in training. Training is drawn first under bucket-balance constraints: a cap on any bucket’s share, scarce buckets kept whole, and roughly 10% of the training budget reserved for the three abstention buckets combined. Development and test splits are then drawn stratified from the remainder, with the test-only ranked-top-k groups reserved directly for `final_test`. Three hard gates execute inside every subsequent data build: no plan may straddle the train/eval boundary, identifiers must be globally unique, and row counts must be conserved. These gates are not ceremony. An early curation bug that double-referenced 1,634 rows was caught precisely because conservation is asserted in the tooling.

Isolation, verified from the frozen artifacts. Train covers 6,674 distinct plan identifiers and `final_test` 2,184; the overlap is **zero**. The split separates *plans*, not merely surface strings: no test question shares its underlying logical program with any training question. At the family level, train carries 37 template-family labels and `final_test` 35, with 34 shared and one test-only family (the ranked-top-k family). That family is retained as a small strict-holdout probe whose results belong in the limitations chapter, not the headline. One residue is disclosed here rather than hidden: an id-level deduplication missed **two verbatim train↔eval duplicate questions** (in the top-k family: identical question text under different identifiers). They are flagged and excluded from final statistics. At two rows in 2,285 their arithmetic effect is negligible, but the lesson is recorded: deduplicate on text as well as identifiers.

Deliberate hardness of the test set. `final_test` is intentionally harder than train, because the benchmark buys statistical power where the thesis makes claims. The *hard-composite* slice is the union of L2 surfaces, the hard subsets (bridge joins, extended operations), the abstention classes, and the hard operators (compare, exists, distinct-set, argmax/argmin, ranked top-k), as implemented in `is_ood_candidate` in the split code. It constitutes 54.7% of `final_test` against 45.0% of train, with the compositional class at 8.4% versus 1.7%. L2 surfaces are 73.3% versus 65.9%, and abstention rows are 15.8% of `final_test` (120 per class) versus 7.7% of train. A terminological rule is enforced throughout this dissertation: although the code field is historically named `ood_candidate`, this slice is **never described as out-of-distribution**. Both train and test contain the class; its plans are disjoint and its surfaces novel, but its *structures* are trained on. "Hard-composite" is the honest name, and the only strict holdout is the single small-n test-only family. Finally, `final_test` flattens the bucket skew: every one of the 13 buckets has at least 93 rows, and top-k has 128, so per-bucket accuracies carry comparable error bars. `dev_tune` deliberately over-samples the development-era hardening targets, which is why it is never cited as an unbiased estimate.

27 Independent oracle verification: the 99.88% audit

A benchmark whose generator also produces its oracles proves nothing about correctness: agreement with the oracle is agreement with the generator. The response is a second implementation (`scripts/qa_independent_eval.py`). It is a pure-pandas evaluator over the raw node/edge Parquet tables that deliberately imports no code from the generator, executor, or reasoning stack. It recomputes every answerable oracle from the flat gold specifications, with its own constraint application, its own six subquery resolvers, and its own implementations of every answer operation.

The first pass agreed on 92.7% of oracles, and the disagreements *clustered by family*. This is exactly the diagnostic value of a second implementation: clusters localise convention gaps rather than random error. Three gaps were diagnosed. First, the bridge-family oracles implicitly assumed the flat first-party record universe and additive-only money aggregation. A to-the-penny reconciliation of a single bridge sum (8,228,406,777.34) proved the convention, which was then made explicit in the gold programs (the 991 additive guards of \$4.2) and the dataset card. Second, the comparison families’ gold programs were incomplete — thresholds, pivot dates and sides lived only in the surface — which motivated the `compare_params` backfill. Third, an empty-string party name was matching every record lacking that party, injecting a constant 1.61-billion excess into every bridge sum on the evaluator side; alongside it, 230 degenerate empty constraints were identified and eliminated. After these repairs, agreement is **14,752 of 14,770 answerable oracles = 99.88%**. The residual 18 are characterised rather than dismissed: 15 in the top-k family, where the independent evaluator must infer ranking parameters, and 3 in one boolean family evaluated by surface parsing. Five comparison-

type families whose parameters are surface-borne are evaluated with declared surface-parse assistance and reported as such.

The claim licensed by this audit is deliberately bounded. 99.88% is *convention-relative agreement between two independent implementations* under the documented conventions (flat first-party universe, additive-only money, latest-release snapshot). It is not a proof of ground truth, and a different convention set would yield different oracles. It is, however, a measured answer to a question benchmarks usually leave unasked: how do you know the oracles are right?

28 The three abstention families as blind-region probes

The benchmark contains three abstention classes, each a first-class design object with 120 rows in `final_test`. **abstain_unsupported** combines a real contract anchor with a concept the knowledge graph does not carry, such as invoice or delivery dates. **abstain_ambiguous** uses a buyer anchor matching many contracts with at least three distinct values of the asked field, so no unique answer exists. **abstain_no_results** pairs a real buyer with a CPV code it has never used, verified to match zero records. Every instance is constructed against the graph: the unsupported concept is checked against the schema, the ambiguity cardinality is counted, and the empty result is executed. Abstention is therefore never a matter of annotator opinion.

Their role in the thesis architecture (Chapter 5) explains why they are traps rather than decoration. The system’s verification chain pins down every property that can be *projected* onto structure and execution. The residue — whether the plan means what the question means — is a blind region that deterministic checks cannot enter, and the abstention layer is what catches it. The three abstention families are the *completeness test of that layer*. Unsupported questions probe whether the system recognises the schema boundary; ambiguous questions probe whether it recognises under-determination; and no-results questions probe whether it can distinguish a legitimate empty answer from a defect (the empty-result trap that naive repair loops fall into). The classes are lexically camouflaged: contrast twins remove word-list shortcuts (§4.2), and diversified rewrites must preserve the cue words, a rule added after three diversified abstention surfaces lost their cues and silently became answerable. A system can therefore pass them only by actually performing the boundary checks. The over-sampling of these classes in `final_test` (15.8% versus 7.7% in train) is the benchmark buying statistical power for precisely this claim.

29 Development/confirmatory discipline and the cue-row disclosure

Midway through system development, an independent adversarial review confirmed a risk this project had created for itself. The pipeline-v2 scaffolding fixes included five new unsupported-cue strings derived directly from `compare_set_v4` misses. Four of the twenty `abstain_unsupported` items in that set are string-matched by them, so their v2 improvement is true *by construction* on those rows. Since `compare_set_v4` is a subset of `final_test`, the contamination could in principle reach the confirmatory set. Three mitigations were adopted and are binding on every results chapter:

1. **compare_set_v4 is relabelled a development set** for all post-v2 claims. Its numbers select models and diagnose failures; they are never cited as headline results. Its only role in this chapter is to define the discipline.
2. **final_test is confirmatory**: evaluated once per system generation, after all fixes are frozen.
3. **Dual reporting**. The five development-derived cue strings string-match 19 of the 2,285 `final_test` rows (0.8%). Every headline number is reported both including and excluding these rows:

Every headline moves by at most 0.21 points; no ranking and no significance verdict changes. One sentence is mandatory whenever this table appears, because without it the negative deltas invite the wrong reading. **The deltas are negative because the 19 cue-matched rows were mostly answered correctly by all systems** — removing near-universally-correct rows lowers every accuracy slightly. The dual report bounds the adaptive-overfitting exposure; it does not reveal an advantage flowing to any one system. Together with the two

Table 4: Benchmark composition: splits, sizes, and roles. `compare_set_v4` (260) is a stratified subset of `final_test` used as the development set.

System	all 2,285	excl. 19 cue rows	delta
teacher	69.76%	69.55%	−0.21pt
Qwen hybrid	78.03%	77.89%	−0.14pt
Qwen fully-local	85.65%	85.57%	−0.08pt
Llama hybrid	75.97%	75.82%	−0.16pt
Llama fully-local	83.33%	83.23%	−0.10pt

disclosed verbatim duplicates (§4.3), this is the complete list of known train/eval contamination channels. Each channel is bounded and reported rather than silently repaired.

30 v4.1 curation and the evaluation-protocol footnote

One further intervention on the frozen test set is disclosed, because frozen-set doctrine forbids silent edits. Objectively malformed question *text* is a data-quality defect, not a difficulty feature, so a v4.1 curation pass was run under four rules. Defects are identified by system-independent text patterns only, never by "questions systems got wrong". The sweep covers the full sets. Only surface text may change; plans and oracles are untouched. The change is versioned with a backup of the pre-edit file. The regex sweep over four malformation classes (raw dictionary literals leaked from the rewrite pipeline, unfilled placeholders, whitespace runs, duplicated-word runs) flagged four `final_test` rows. Of these, **two were fixed** (a trailing resolver-dictionary artifact stripped from a bridge question; a duplicated year collapsed in a date question) and two were adjudicated as legitimate English; `compare_set_v4` had zero hits. Integrity checks confirm 2,285 rows with identifiers and oracles byte-identical and only the two question strings changed. Since nothing had yet been evaluated on `final_test`, no result mixes versions. The impact bound is ≤ 0.09 points. This also closes an argument: the difficulty of `final_test` comes from design (the hard-composite share, L2 surfaces, and abstention traps), not from residual data noise.

A final footnote separates two configurations that must never be blended. The *evaluation protocol* used for every cited accuracy runs with a repair budget of one. The *data-engine harvest* of Chapter 6 deliberately runs with a repair budget of two, because a harvester should extract as much verified supervision as each question allows. "Data-engine yield" and "system accuracy" are therefore different metrics under different budgets, and no table in this dissertation compares a number of one kind against a number of the other.

In sum, the benchmark’s authority rests on four measured properties: executable gold programs, dual-implementation oracle agreement of 99.88% with the residual 18 characterised, plan-level split isolation verified at zero overlap, and abstention traps that test the system’s blind region. It also rests on a use discipline that quarantines development feedback loops and dual-reports the one bounded exception. Chapter 5 now describes the system that must answer these questions, and the results chapters draw on the guarantees established here.

31 System: Hard Verification, Abstention, and the Movable Projection Boundary

Chapter 3 built the knowledge graph, and Chapter 4 built the benchmark that measures question answering over it. This chapter describes CICADA, the system that answers those questions over the same graph. Its central idea is a strict division of labour: a language model proposes, deterministic code checks and executes, and the system abstains where neither can support an answer.

32 Design goal and the three-layer principle

CICADA’s design goal follows from the constraint of Chapter 1: an answer is only ever produced by deterministic execution against the knowledge graph, and where the system cannot support an answer it must

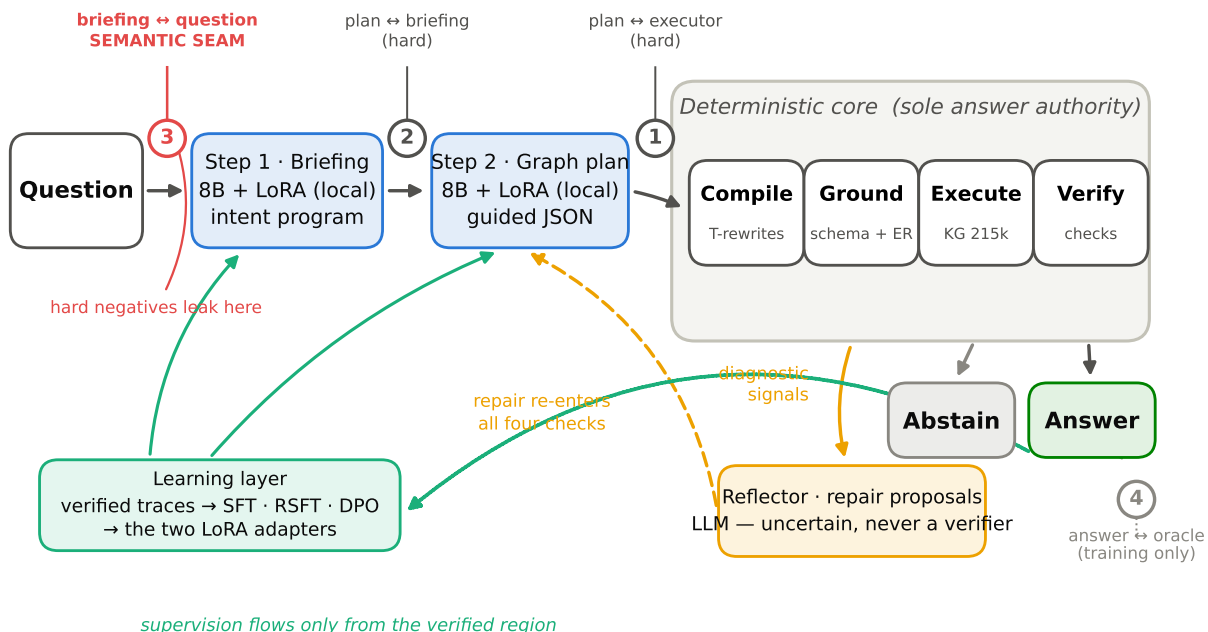


Figure 1: The CICADA architecture. A question passes through two local LoRA planners (Step 1 briefing, Step 2 typed graph plan) into the deterministic core, the sole answer authority. Circled numbers mark the four-check chain; check 3 (briefing↔question) is the semantic seam where hard negatives leak. The reflector consumes diagnostic signals and its repairs re-enter all four checks. Verified traces alone supervise the two adapters (learning layer).

abstain. Between question and executed answer lies a chain of semantic judgements: what kind of answer is wanted, which literals constrain it, which organisation plays which role, and whether the concept asked about exists in the graph at all. A language model is indispensable for making these judgements and untrustworthy for certifying them. The architecture therefore asks one question of every property on that chain: *can it be projected onto structure or execution?* If it can, it is pinned by a deterministic check, and the model is never trusted for it. If it cannot, it falls into a blind region that no mechanical check can see into. The response to that region is twofold. Calibrated abstention catches what projection misses, and learning — trained only on signal from the verifiable region — gradually covers part of what remains.

This yields the three layers that structure the chapter. The **hard-verification layer** (§5.3–5.4) pins every projectable property: compilation, grounding, execution, and structural cross-checks between question, briefing, and plan. The **abstention layer** absorbs the unprojectable residue; its completeness is tested by the benchmark’s three abstention-trap families (ambiguous, schema-unsupported, and empty-result questions; Chapter 4). The **learning layer** (Chapter 6) consumes only what the first layer certifies. The orchestrator records this division of authority in every trace it emits, as a literal string in the workflow metadata: *"LLM proposes; KG executes; verifier judges; reflector refines"* (`pipeline.py`, `_workflow_trace`). Two further claims give the chapter its shape. First, the boundary between the projectable and the unprojectable is *movable*: §5.6 documents one move, the v2 revision, including the measured cost of moving it too far. Second, the reflector that repairs failed plans sits deliberately *outside* the verification chain. It consumes verifier signal but never joins the chain, and every repair it proposes is re-checked by that chain (§5.5).

33 Two-step planning: the language model as translator

The language model appears in exactly one role: translating the question into a machine-checkable intermediate representation. It never sees the graph’s contents, never composes an answer, and never adjudicates

its own output. Everything it emits is a *claim about the question*, expressed in a vocabulary small enough for deterministic code to audit.

Step 1: the intent-program briefing. The first call (`question_intent_program_messages`, `typed_planning.py`) asks a reading model for a structured *briefing*. The briefing contains an `answer_signature` (operation, value type, natural-language answer-field description), a `program` of at most eight typed steps whose operations come from a closed enumeration (`filter_records`, `bind_filter`, `distinct`, `count`, `sum`, `select`, `exists`, `compare`, `argmin/argmax`, `top_k`, `abstain`), an `answer_step`, and an `unsupported_or_ambiguous` list. The prompt’s hard rules are contractual: root steps take no inputs (the model may not invent a universe table); literals must be copied verbatim, never invented; and an unsupported or ambiguous question must be expressed as `operation=abstain` with an empty program. Filters describe fields in natural language, such as "published year" or "buyer organisation". The mapping to schema columns is left to the deterministic grounding layer (§5.3). A well-formed intent program takes a **fast path**: it compiles deterministically into a graph plan without a second model call, subject only to a narrowly targeted veto (`_fastpath_veto`, §5.6). Malformed programs are not reinterpreted. A contract verifier (`_verify_intent_program`) checks every step’s operation against its declared return type, every input against earlier steps, and the answer step against the signature; violations become structured failure codes rather than guesses.

Step 2: the typed graph plan. When the fast path is unavailable, a second call (`typed_plan_messages`, lean variant) turns the question plus briefing into a `graph_plan`. This is a flat array of variables — each with a kind (`entity`, `entity_set`, `record_set`, `scalar`), a role, and filters over an enumerated slot vocabulary — plus relations with typed bind fields and an enum-closed `return` object. Variable identifiers encode structure (`a1` feeds the answer, `b1a1` feeds `a1`), so the dependency DAG is readable from a flat array even when the model omits explicit edges. Prompt and schema variants are resolved per model from measured ablations (`resolve_planner_variants`); the local 8B students take the lean prompt with the omit-unused schema.

Guided decoding as format authority. Both steps are shape-enforced by strict JSON schemas (`intent_program_schema`, `graph_plan_schema`) passed to the provider or, for local models, to vLLM’s guided decoding. Enumerations do real work here: slot names, operations, and roles are enum-constrained, which eliminates placeholder echo and invented vocabulary at the decoding level. Format is therefore externally guaranteed for every system evaluated, including the untrained zero-shot rung. Differences along the training ladder measure planning quality, not format compliance; §5.7 gives the diagnostic that makes this point empirically.

Structural resampling. The evaluation configuration runs `plan_samples=2`. If the first sample fails compilation or the consistency check — a *detectable* defect with a structured reason — the planner requests one fresh, deliberately different decomposition before falling back to abstention or repair. The boundary is principled: an executable plan that is semantically wrong is never resampled, because at inference time no oracle exists to detect it.

34 The deterministic core: compile, ground, execute, attest

Everything downstream of the planner is deterministic code, and it is deliberately the largest part of the system.

Compilation and structural repair. `compile_graph_plan` (`graph_planning.py`) converts the model’s graph JSON into an executable plan. It derives dependencies from the identifier convention and treats relations as dataflow edges, because a relation without a matching dependency would otherwise silently never bind; explicit edges win over naming-derived ones when they conflict. Before compilation, `normalise_graph_plan` applies a catalogue of *T-transforms*: deterministic rewrites of mechanically recognisable planner idioms into the structure they evidently mean, never guessing semantics. Among them: a filter whose value names another variable becomes a dependency edge (T1). An empty-valued filter is schema padding, never executable, and is dropped (T2). A compare side that is a scalar variable holding one literal amount or date is folded into the return as a threshold, rather than executed as a spurious record filter (T11/T12). An echoed bridge anchor (the organisation both bound and literally re-filtered) is stripped so the

bind carries the meaning (T9). A singular interrogative ("which buyer...") answered with `distinct_set` is rewritten to `select`, so run-time uniqueness adjudicates instead of a 30-item list hiding an ambiguity (T10).

Compilation itself enforces DAG validity, with a second-chance compile that drops naming-derived edges when they alone create a cycle. It rejects an *unconstrained bind source*: an entity set with no filters and no dependencies is the whole dimension, and binding on it would silently answer a different question. It also grounds *dangling* variables, so an unsupported concept smuggled into a never-executed variable still fails the plan with its reason intact. Forms the single-query executor cannot express are rewritten. A two-organisation comparison becomes a `DecompositionPlan` of two independent sub-counts combined by `compare_gt`; `rank_top_k` maps onto the grouped `top_k`; and `find_extreme` maps onto `argmax/argmin` over the exhaustive match set, answering with the extremum *record* (its `contract_node_id`) rather than a bare value.

Schema grounding. Natural-language field descriptions from Step 1 map to canonical slots via a two-stage grounder (`schema_grounding.py`). Candidates are generated over an alias table plus a character n-gram embedding; validation then applies a type gate, a confidence threshold of 0.72, and a top-2 margin of 0.08. An ambiguous match is a structured failure, not a coin flip. Two asymmetries matter. First, a direction cue outranks the role noun: "buyer has awarded a contract to X" contains *buyer*, but the "awarded ... to" phrasing names X as the supplier, and the cue wins. Second, a field text containing a token from the unsupported-concept lexicon (`_UNSUPPORTED_FIELD_TOKENS`: invoice, payment, delivery, performance, reliable, bidder, fairness) is denied the permissive substring shortcut. "Invoice or payment date" can therefore no longer ground to the date slot and be answered from the award date — the v2.2 rule whose calibration history §5.6 recounts.

Entity resolution. Organisation surfaces resolve under a deliberately conservative policy (`resolve_confident_org`). The top candidate is accepted only if it is an exact record match, or if it scores at least 0.85 with a margin of at least 0.10 over the runner-up. Near-ties are inspected for *decoration variance* — "The X", "X (BEIS)", ampersand and case variants of one organisation. Because the organisation's rows are split across those surfaces, such a mention is resolved as an `in` filter over every variant rather than a silent pick of one. Genuinely distinct near-ties fail as `ambiguous_entity_candidates`. No caller anywhere takes `hits[0]` under a weak match.

Spec grounding. Immediately before execution, `ground_spec` (`grounding.py`) rebuilds every field, operation, and guard of the proposed query. The executable operation set is closed (`select_unique`, `count`, `sum`, `exists`, `argmax`, `argmin`, `distinct_set`, `set`, `top_k`, `predicate`); requests outside it fail with a stated reason (averages unsupported; `compare` and `in_subquery` routed to the decomposition planner). Field aliases snap to canonical columns. Mechanically impossible value shapes are rejected *because they would otherwise succeed*: an `in` filter over free text, or a bare year against a timestamp column, executes "successfully" and returns a verified-looking zero. Every money aggregation — a sum, a predicate over a sum, a `top_k` ranked by summed value — has a `value_is_additive` guard added if the planner omitted it, so a total never silently includes non-additive framework-ceiling values. Every reduction is forced onto exhaustive retrieval.

Execution. The executor (`executor.py`) runs the grounded spec over the full knowledge graph, never a retrieved sample, after preflight checks on schema fields, constraint contradictions, and exhaustiveness. Match sets are deduplicated on `contract_node_id`. `count` and `exists` answer over the possibly empty match set, so 0 and False are legitimate verified answers. `select_unique` returns `multiple_answers` when the match set disagrees, which becomes a clarification rather than an enumeration. `top_k` grouping excludes empty-string keys, so unnamed populations cannot occupy a rank. Monetary arithmetic is exact `Decimal` arithmetic with non-finite values barred. Evidence is attested exactly: the bundle carries the exact match count, with row identifiers capped at 200 and the cap flagged, so an audit never confuses a sample with a population.

Evidence and answer cards. A passed execution is wrapped in a KG-first evidence verdict (`evidence.py`) binding the claim to evidence identifiers. Post-flight disclosures never change the answer but make it self-explaining: a count discloses how many matched contracts lack a recorded supplier, and a factoid confirmed across *n* agreeing contracts reveals hidden multiplicity. A failed or unsupported execution produces an abstaining answer card carrying the structured reason. Abstention on emptiness is *provenance-gated*: an

empty result whose every literal traces to the question is treated as the graph's answer, a rule enforced at the repair gate (§5.5).

35 The four-check chain and the seam

The machinery of §5.3 can be summarised as four checks on the correspondence chain from answer back to question. Their character differs, and stating the difference precisely is the centre of this chapter's argument.

Check 1 — executor $\leftrightarrow$ plan graph: hard. The answer is produced *by* compiling and executing the plan. The correspondence between what the plan says and what the executor does holds by construction, deterministically, on every question.

Check 2 — plan graph $\leftrightarrow$ briefing: hard. The briefing's answer signature, declared template, and program structure are compared against the compiled plan in code. The intent-program contract verifier rejects an answer step whose operation or return type contradicts the signature, and the consistency check rejects an operation outside the declared question type. This is structural comparison, exhaustive over what it covers.

Check 3 — briefing $\leftrightarrow$ question: the seam. Here code can check only *projections* of meaning, never meaning itself. `plan_consistency_check` audits every projection the question text affords. Literal filter surfaces must occur in the question (`surface_not_in_question`). Every number in a constraint or threshold must be question-supported, with deterministic decompositions for scaled amounts ("£2.5m" supports 2500000) and written dates (`invented_number`, `invented_threshold`). Buyer/supplier roles are checked against direction-cue windows around the mention, with bridge anchors exempted (on "suppliers who worked with X" the plan *correctly* constrains X as buyer) and interrogative answer-role phrases masked (`role_flipped`). Explicit atoms (years, CPV codes, categories, quoted titles) must survive into the compiled spec (`missing_explicit_constraint`). Strong bridge phrasings on a genuinely flat plan flag a collapsed intermediate set (`bridge_cue_requires_bridge_join`, fired only when the graph carries no dependency structure). Comparison direction words must support the plan's comparator. A cue lexicon of concepts the graph does not carry forces the unanswerable reading. A record-count question whose return edge counts an `entity_set` variable is rejected as the count-intermediate-set misread (§5.6). Each of these is a hard check *of a projection*. None of them, and no possible extension of them, checks whether the plan means what the question means. Check 3 is therefore the entrance to the blind region, and exactly here **hard negatives** arise: plans that pass every projection, ground, execute, and verify cleanly, and are nevertheless wrong. When the training harvest later compares verifier-passing outputs against the offline oracle, the verifier-passed-but-oracle-wrong residue is exactly the population that leaked through this seam. Those specimens are collected as `hard_negatives`; their use and misuse as preference-learning material is the subject of Chapter 7.

Check 4 — answer $\leftrightarrow$ oracle: hard, but training-time only. During data harvest, and only there, a fourth deterministic check compares the executed answer against an independently computed oracle. The oracle filters candidate training examples; it never authors them. It does not exist in the inference pipeline, and it must not: consulting it at evaluation time would be cheating, and every number reported in this dissertation is produced without it.

The framing this dissertation commits to has a negative half that matters as much as the positive half. Everything hard-verifiable is hard-verified; what cannot be hard-verified is compressed to a minimum; abstention and learning catch the remainder. The four checks squeeze the blind region layer by layer, but they do not eliminate it.

36 Gated repair: the reflector as a consumer of verifier signal

When a check fails, the failure is not the end of the trace. A second language-model component — the *reflector* — receives it and proposes a repaired plan. What the reflector is not is essential: it is **not a fifth check**, and this dissertation never describes it as a verifier. The four deterministic checks *produce* diagnostic signal. The reflector, an uncertain model, only *consumes* that signal to propose a repair, and the proposal re-enters the same four checks. The reflector never certifies its own fix: a repair is accepted only when the

full ground $\rightarrow$ execute $\rightarrow$ verify path passes it. The design deliberately refuses the alternative, an uncertain model verifying an uncertain model, because that pattern invites an endless regress.

The signal the reflector consumes is rich and strictly answer-free. `_feedback_from_trace` assembles the failure stage (grounding, schema, executor, sanity, verifier, or plan-compile), the structured reason, the failed plan with its raw graph surfaced even on compile failures, the failed variable, and grounding issues with top candidates. A scrubber strips anything oracle-shaped before any model sees it. A deterministic reason-to-prescription table (`_REPAIR_GUIDELINES`) prepends targeted guidance: a cycle diagnosis names which edge direction to keep, and `no_results_with_invented_literals` says to remove exactly the literal the question never stated. The reflector answers under its own strict schema: a diagnosis, one action from a closed set of six (`fix_question_type`, `repair_constraints`, `swap_buyer_supplier`, `change_operator`, `change_operation`, `abstain`), the repaired plan, and explicit changed/unchanged field lists.

Repair is *gated* before it is attempted. The eligibility rule (`_replan_eligibility`) separates a repairable defect from a legitimate abstention. A plan that never executed, an executor error, or a `no_results` caused by a planner-invented literal may be repaired. But a ***clean no_results* — grounded, executed, every user-visible literal traceable to the question — is the knowledge graph’s answer**, and repairing it is answer-shopping. The provenance test (`_invented_literals`) excludes bound values, invisible guards, and long derived in-lists, so only genuinely invented user-facing literals open the gate. The motivating measurement comes from the ≤ 50 -question development smoke slice and is cited as directional only. A naive "no answer $\rightarrow$ replan" loop *reduced* accuracy from 84% to 66% and raised hallucinated answers from 1 to 6, repairing six correct abstentions into confident wrong answers. The gate exists to make repair monotone.

The loop is bounded and fully recorded as an **attempt protocol**. Attempt 0 is the initial plan; each subsequent attempt consumes the *latest* failure’s feedback; and every attempt’s plan, status, and feedback lands in `metadata["feedback_replan"]["attempts"]`. This record is the substrate of the learning layer. A harvest run computes Repair@k from it and mines preference pairs directly (chosen = the first verified attempt, rejected = the nearest preceding failure), which is where the DPO training data of Chapter 6 originates. Two repair budgets exist as deliberately distinct configurations, never to be conflated. The **evaluation protocol** runs `max_feedback_replans=1` (`run_compare.py`), so every ladder rung is scored under the same one-repair budget. The **harvest protocol** (`run_teacher.py`) runs `-max-repairs 2`, spending a larger budget to maximise verified yield. Evaluation numbers are never numbers "under the harvest config", and the two metrics they feed — system accuracy versus data-engine yield — are kept apart throughout.

Harvest routing completes the interface to the learning layer. Verifier-and-oracle-passed traces become `verified_sft`; verifier-passed-but-oracle-wrong traces become `hard_negatives` (the seam’s leakage made visible); correct abstentions become `abstain_sft`; successful repairs become `repair_sft`; and the attempt protocol yields preference pairs. Chapter 6 takes over from there.

37 The movable projection boundary: the v2 revision as a worked example

The blind region is not a fixed set. The seam is where projection *currently* stops, and engineering effort can move it: a property that today only the oracle can see may tomorrow be a structural check. The v2 revision shows this in practice, in both directions.

The revision began from an error autopsy of the v1 development matrix. `compare_v4` is the development set; every fix below was derived from its v1 errors and confirmed only later on the untouched final test set. The autopsy found 21 abstentions on answerable questions, 12 answers on unsupported questions, and 22 wrong values, root-caused to four precise defects. Each fix is a projection move: a semantic error class re-expressed as a structural property.

1. **The count-target check.** The dominant verifier-invisible bridge failure was the *count-intermediate-set* misread. On "how many contract notices..." questions, the plan counted the intermediate entity set instead of the target records; in the motivating trace, 30 CPV codes where the oracle counted 1,439 notices. Such a plan is semantically wrong, structurally flawless, and invisible to execution. `_count_target_is_entity_set` makes the defect structural: a record-count ques-

tion whose return edge counts an `entity_set` variable is rejected before execution. The trigger is deliberately precise — "how many CPV codes/suppliers" does not match the record-count pattern.

2. **The empty-value type-gate exemption.** Categorical lookups ("what is the procurement category of...") ground a field text with an *empty* value. v1's type gate rejected them for failing the goods/services/works enumeration, silently unmapping every categorical lookup's return field. The v2 gate exempts the empty value, because a lookup context has nothing to validate, while still rejecting any non-enum *stated* value.
3. **The fast-path targeted veto.** The intent-program fast path (§5.2) bypassed Step 2 entirely, and with it the two consistency checks that catch failures execution cannot: the unsupported-cue check and the count-target check. `_fastpath_veto` applies exactly those two checks to compiled fast-path graphs — and, deliberately, *only* those two.
4. **The unsupported-field-token rule.** Alias grounding's permissive substring rule let "invoice or payment date" ground to the date slot at high confidence, so an unsupported-concept question was answered from the award date. The v2.2 rule (§5.3) blocks the substring shortcut exactly when the field text names a concept from the unsupported lexicon.

The iteration was not clean, and the record is kept honest. The first attempt, v2.0, ran the *full* consistency battery on fast-path graphs. Those checks are calibrated for raw planner payloads, and on compiled graphs they false-positive: the teacher's development leg regressed several points, with answerable questions flipping to null on cue-free wording. v2.1 narrowed the gate to the two-check veto above. The second over-reach was subtler. v2.1 replaced the substring alias rule with token-coverage scoring, which passed the entire synthetic unit battery — and then rejected **88% of 169 real field texts** harvested from actual teacher-trace briefings. Real field texts are sentences ("Was the award signed after 1 May 2025?") that v1's substring rule had grounded without difficulty. The teacher's factoid bucket collapsed, while the students, whose trained plans carry canonical slots, were unaffected. v2.2 reverted to v1's permissiveness *except* for the unsupported lexicon, and was verified decision-identical to v1 on all 169 real field texts while still rejecting the unsupported-concept phrasings. Two lessons are recorded. First, **over-projection wounds the system exactly as under-projection does.** A check that fires on legitimate traffic converts answers into abstentions, just as a missing check converts abstentions into hallucinations. Moving the boundary therefore demands regression against *real pipeline intermediates*, not synthetic unit cases. Second, single-run teacher deltas are treacherous. The subsequent three-replicate noise floor (v2.2 teacher $72.6\% \pm 1.1$, replicates 71.9/71.9/73.9, against a v1 single run of 70.0%) showed that part of the v2.0/v2.1 regression chase was provider noise. Single-run teacher differences below about three points are therefore treated as unclaimable. The v2.0 and v2.1 intermediate matrices are archived as non-citable; only v1 and v2.2 are citable pipeline versions.

The measured effect of the boundary move. On the development set ($n=260$), $v1 \rightarrow v2.2$ lifts every rung of the ladder by roughly five points: zero-shot $61.5\% \rightarrow 70.4\%$, SFT $76.1\% \rightarrow 81.2\%$, RSFT $76.1\% \rightarrow 81.5\%$, DPO $78.8\% \rightarrow 83.5\%$. The reading discipline is strict. This is a *scaffolding* effect — the same adapters, a raised floor for every system including the untrained zero-shot rung and the teacher — and it must never be reported as a training gain. Training gains are measured on top of the raised floor, and Chapter 6 owns that decomposition. What the five points measure is the value of one boundary move: formerly semantic error classes are now caught structurally, before execution, for every system that runs the pipeline. What could not be carried across the line, plan-versus-question meaning in the general case, remains with abstention and with learning.

38 Serving: one GPU, guided decoding, fully local

The deployment target motivates the final design decisions. Each student stage serves from a single local GPU via vLLM's OpenAI-compatible server (`serve_student.sh`). The base model loads once, and fine-tuned stages are hot-plugged as named LoRA adapters (`-enable-lora -max-lora-rank 64`) with no weight merging. The zero-shot rung and every trained rung therefore share identical base weights and serving configuration (Qwen3 in non-thinking mode, matching its training distribution). Format enforcement is uniform at

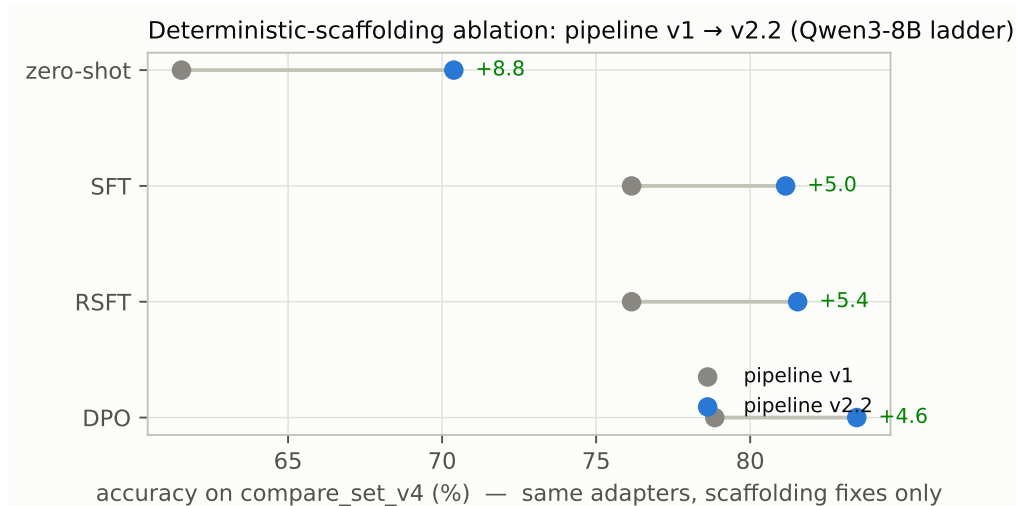


Figure 2: Deterministic-scaffolding ablation: the same four adapters evaluated under pipeline v1 and v2.2. Every rung gains; the adapters are unchanged.

request time: clients send `response_format=json_schema`, which vLLM maps to guided decoding, so *every* ladder rung — including the untrained base — is format-locked, and rung-to-rung deltas measure planning skill rather than JSON discipline. In the fully-local topology, the evaluation driver points Step 1 at a second local vLLM endpoint (`-step1-base-url`), and no request of any kind leaves the machine. The cost, control, and privacy objections of Chapter 1 are answered by construction rather than by policy.

The schema diagnostic: what fine-tuning actually teaches. One measurement seals the chapter’s argument about where competence lives. With guided decoding *disabled* (n=100, development set), the untrained Qwen3-8B base parses as JSON on 100/100 questions. It produces plans grounded in the question (real entities, real years, real CPV codes, recognisable filter structure) on **98/100**. But it conforms to the compiler’s target contract on only **3/100**. Opening the raw outputs (deliberately, not trusting the gate) shows the base emitting question-grounded plans in a fluent *self-invented* schema — `op:filter_records`, `args.filters`, `id/inputs` — that the deterministic compiler cannot consume. The SFT and DPO students conform on 98 and 99 respectively. The conclusion is sharp. Fine-tuning does not teach planning ability, because the base already extracts entities and builds filter structure. It teaches **conformance to the specific executable contract** (3 → 98/99). And since guided decoding enforces exactly that contract at evaluation time, the accuracy ladder isolates planning *quality within the contract*, not the ability to emit it. Two recorded caveats bound the claim. The diagnostic stored only failing raws, so trained-arm passes were not individually inspected; the claim rests on the base arm’s verified grounded-but-off-schema pattern. And one base abstention expressed through `return.operation` was under-counted, to which the 3-versus-98 direction is insensitive. The diagnostic itself survived three prior opposite-signed gate flaws, each caught by raw inspection; that audit trail belongs to the evaluation methodology of Chapter 4.

The three layers can now be restated as what the code does. The hard-verification layer pins every projectable property (checks 1–3, with the oracle confined to training time). The abstention layer catches what projection cannot see, gated on provenance so that emptiness is an answer. The learning layer consumes only what the first layer certifies: the oracle-filtered, never oracle-authored, training pools whose consequences the next chapter measures.

39 Bootstrapping Experiments: Ladder, Yields, and the Five-Pairing Confirmation

Chapter 5 built and hardened the pipeline. This chapter trains models inside it and measures the result. The claim is that the learning layer of the architecture works, and that it works three times over, in three progressively stronger forms. First, teacher-filtered distillation: an 8-billion-parameter student trained only

on verifier-passed, oracle-gated teacher outputs climbs a four-rung training ladder on the development set. Second, verifier-gated self-improvement: the student replaces its teacher inside the same harvesting harness, and its rejection-sampled self-harvest yields more verified, oracle-correct training data than the teacher’s harvest did. Third, Step-1 denoising distillation: briefings filtered by whole-pipeline outcome produce a fully-local two-stage system that runs without any cloud model. The confirmatory evidence is a single held-out scoreboard on the frozen test set ($n = 2,285$): teacher 69.76% < Llama hybrid 75.97% < Qwen hybrid 78.03% < Llama fully-local 83.33% < Qwen fully-local 85.65%. Every adjacent and non-adjacent pairing is significant under McNemar’s test at $p < 10^{-14}$. Throughout, supervision flowed only from the verifiable region of the task; the resulting systems still surpass the source of that signal.

40 Experimental design: two configurations, one metric discipline

All experiments use the benchmark of Chapter 4. Its 12,828 questions are split into a training pool (9,267), development slices (dev_tune 556, dev_select 671, dev_smoke 49), and a frozen confirmatory test set, final_test (2,285). Model selection throughout this chapter is done on compare_set_v4, a 260-question development set stratified as 13 buckets of 20 questions. final_test is touched exactly once per selected system, after all development decisions were frozen. Because compare_set_v4 was drawn as a stratified subset of final_test, development results are reported only in a model-selection role and never as headline numbers; the headline lives on final_test alone.

Two distinct pipeline configurations appear in this chapter, and they must be kept apart to read the tables correctly.

The **harvest configuration** is a data engine. It is implemented by `scripts/run_teacher.py`, which runs the full live two-step pipeline over the training pool. A Step-1 understanding model produces a dense briefing, and a Step-2 planner produces a typed graph plan. The plan then passes through deterministic grounding, compilation, exhaustive execution over the knowledge graph, and the verifier chain. The harvest runs with a repair budget of two (`-max-repairs 2`). Every verifier-rejected plan is fed back, with structured failure feedback, for up to two gated replanning attempts, each re-run through the complete ground–execute–verify path. The goal of this configuration is to maximise the yield of verified, correct training material. Its output metric is **data-engine yield**: the fraction of questions for which the engine produced a verified plan whose answer also matches the held-out oracle.

The **evaluation configuration** is a measurement instrument. It is implemented by `scripts/run_compare.py` and orchestrated over the ladder by `scripts/eval_ladder.sh`. It holds a deliberately tighter budget: repair budget one (`max_feedback_replans=1`), two plan samples per question, and schema-guided JSON decoding on every rung, including the untrained zero-shot rung. This keeps all cells of the evaluation matrix comparable under an identical inference budget. Its metric is **system accuracy**: type-aware exact-match against the oracle on answerable questions, with an unanswerable question scored correct if and only if the system abstains, exactly as defined in Chapter 4.

Yield and accuracy are different quantities, measured on different question sets under different budgets, and this chapter never compares one against the other. The teacher appears twice in what follows, at 65.5% as a data engine and at 69.76% as an evaluated system, and these are statements about two different things.

Training uses a single recipe across all adapters and both base models, so every rung-to-rung and base-to-base comparison is a data comparison rather than a hyperparameter comparison. All adapters are QLoRA fine-tunes (4-bit NF4 quantised base, LoRA rank 64, alpha 128, dropout 0.05, all linear targets), trained with LLaMA-Factory in bf16 with SDPA attention, sequence cutoff 6,144 tokens, and an effective batch size of 16 in every stage. Stage-specific settings are as follows. SFT runs at learning rate 1×10^{-4} for 3 epochs with cosine decay. Rejection-sampling SFT (RSFT) continues the SFT adapter at 5×10^{-5} for 2 epochs. DPO continues the RSFT adapter at 5×10^{-6} for 1 epoch with sigmoid preference loss and $\beta = 0.1$ (`configs/training/*.yaml`). The two bases are Qwen3-8B (served and trained without thinking blocks, matching its serving configuration) and Llama-3.1-8B-Instruct.

A final design invariant concerns the training data itself. The exporter (`scripts/export_llamafactory.py`) renders every training prompt with the *runtime* prompt builders — the same `typed_plan_messages` and

`typed_replan_messages` functions the deployed pipeline calls — so the distribution the student is trained on is byte-identical to the distribution it is served on. Targets follow a single serialisation convention: compact JSON of the compiler-normalised graph plan. To prevent shape memorisation, the exporter caps emission at 150 samples per template family and 400 per question bucket. The count bucket spans roughly eight families and would otherwise make up about a third of the pool, teaching a "when unsure, count" prior. Abstention examples are capped at 10% of the verified plan pool, because uncapped teacher pools run near 14% abstention against a natural rate of roughly 5% and would over-teach refusal.

41 The data engine: teacher harvest over the training pool

The first harvest ran the live teacher — GPT-5.4-nano as Step-1 and Grok-4.1-fast as Step-2, at temperature 0 — over all 9,267 training questions. Acceptance into the supervised pool is doubly gated. A plan must pass the full deterministic verifier chain, and its executed answer must also agree with the held-out oracle (a mechanical answer-shape gate rejects, for example, a numeric answer to a boolean question). The oracle's role is strictly that of a filter. It never contributes an answer, a plan, or any content to a training target; it only decides whether a pipeline-produced artifact may enter the pool. Verifier-passing plans whose answers fail the oracle gate are the most valuable rejects: they are exactly the plans the runtime verifier cannot distinguish from correct ones. They are routed to a hard-negative pool and offered one oracle-*gated* repair attempt, in which the feedback discloses only that the answer "failed external validation", never the oracle content itself.

Each question therefore lands in exactly one of several sinks. `verified_sft` holds verified and oracle-correct plans, with briefing and question. `abstain_sft` holds correct abstentions on unanswerable questions, teaching the abstain plan shape as a first-class success. `repair_sft` holds feedback-conditioned repair demonstrations whenever success arrived on a repair attempt. `dpo_pairs` holds the successful attempt as *chosen* against the nearest preceding failed attempt of the same question as *rejected*; we call this the attempt protocol. The remaining sinks are `hard_negatives` and `failures`. One point about the attempt protocol deserves emphasis: preference pairs are not synthesised. They are the repair loop's own before/after trajectories, so both sides of every pair are plans the pipeline actually produced for that question.

On the answerable portion of the pool the teacher's data-engine yield was **65.5%** (5,605 of 8,555 verified and oracle-correct; `data/qa/teacher_full_v1`, recomputed from traces), with correct abstention on 86.8% (618/712) of unanswerable questions. Of 6,860 verified plans, 4,788 were verified on the first attempt, so the repair loop contributed roughly two thousand additional verified plans. The repair budget pays for itself at harvest time even though, as Chapter 5 established, it is roughly accuracy-neutral at inference time. Routing produced 5,598 verified-SFT rows and 590 abstention rows, alongside 1,262 hard negatives, 1,725 repair demonstrations, and 390 preference pairs. After family and bucket caps, the exported round-1 dataset comprised 3,632 plan-SFT training samples (76 validation), 1,679 repair samples, and the 390 pairs.

Two disciplinary notes attach to these numbers. First, 65.5% is a yield, not an accuracy. It was obtained under the harvest budget on the deliberately hard training pool, and it is not comparable to any number in Sections 6.7–6.8. Second, the pool is hard by construction. Its composition is dominated by the difficult buckets (count, bridge_join, factoid), precisely so that the harvest concentrates supervision where the task is hardest. Per-bucket usable yield ranged from above 92% on count and sum questions down to 24.9% on bridge_join, which consequently doubles as the richest source of hard negatives.

42 Supervised fine-tuning: the first rung on both bases

The SFT rung trains each base on the exported teacher pool (plan-SFT plus repair-SFT). Both runs were uneventful in the best sense: 840 optimisation steps over 3 epochs, roughly two and three-quarter hours of wall-clock per base on a single H100. Validation loss on the held-out plan slice decreased monotonically (Qwen 0.0186 at step 200 to 0.0128 at convergence; Llama 0.0181 to 0.0118), and the two bases' curves did not diverge. Each adapter was smoke-tested on a small stratified slice behind a vLLM endpoint before entering the evaluation matrix; per the pre-registered discipline, smoke figures are sanity checks and are not reported as results.

Table 5: Data-engine yields across bootstrap rounds (oracle-correct on answerable train-pool questions; a harvest metric, not system accuracy).

Data engine (Step-2 policy)	Sampling	Answerable yield	Correct abstention
Teacher (Grok-4.1-fast)	greedy	65.5% (5,605/8,555)	86.8% (618/712)
Qwen SFT student	temp 0.7×4	76.6% (6,556/8,555)	88.1% (627/712)
Llama SFT student	temp 0.7×4	78.1% (6,685/8,555)	—

Under the evaluation protocol on the development set (pipeline v2.2), SFT lifts Qwen from a zero-shot 70.4% to **81.2%**, and Llama from 60.0% to **83.1%**. The jump is larger on the weaker base. This is consistent with the interpretation, developed in Chapter 7, that fine-tuning teaches conformance to the executable planning contract rather than planning ability itself. The full matrix is presented in Section 6.7.

43 Rejection-sampled self-harvest: the student out-harvests its teacher

The central bootstrapping question is whether the student can replace the teacher *inside the data engine*. The RSFT round answers it. We re-ran the harvest harness of Section 6.2 with a single substitution: Step-2 was pointed at the local SFT student (served via vLLM) instead of the cloud teacher. Sampling ran at temperature 0.7 with four plan samples per question, so that rejection sampling can explore beyond what greedy decoding already solves. Step-1, the repair budget of two, the oracle gate, and every sink remained identical.

Table 6.1 — Data-engine yield on the training pool (harvest configuration, repair budget 2; oracle-correct on answerable, $n = 8,555$). Yield figures; not comparable to system accuracy.

Both students exceed the teacher’s yield by a wide margin: +11.1 points for Qwen and +12.6 for Llama. Abstention quality does not degrade under temperature sampling. This is the first exceeds-teacher evidence in the thesis, and its scope must be stated carefully: it is a statement about the *data engine*, not about model accuracy. Temperature-0.7 rejection sampling with four candidates, a deterministic verifier gate, and a two-repair loop finds correct plans beyond both the teacher’s greedy decoding and the student’s own. The Qwen run’s verified count rose from 4,712 at the first attempt to 7,601 after sampling and repair, a gain of 2,889 verified plans that comes from the search-and-filter machinery rather than from the policy alone. The eval-accuracy comparison between student and teacher is a separate claim, with its own numbers and its own question set (Sections 6.7–6.8).

The honest flags belong in the text rather than a footnote. On the bridge_join bucket the Qwen self-harvest *verified* 86.7% of questions but was oracle-correct on only 52.3%. This 34.4-point band of verifier-passing-but-wrong plans is the signature of the check-3 blind region described in Chapter 3: the count-intermediate-set misreading is structurally valid and executes cleanly. Exactly as designed, this band did not contaminate the supervised pool — the export remains oracle-gated, and all 6,550 exported verified-SFT rows are both verified and oracle-correct. Instead the band was converted into fuel: 1,051 hard negatives and 689 new *on-policy* preference pairs, against the teacher round’s 390. Even at 52.3%, the student’s bridge yield roughly doubles the teacher’s (~25% usable on the same bucket).

Training on the self-harvest (RSFT: continuing the SFT adapter at 5×10^{-5} for 2 epochs on 3,084 plan and 3,169 repair samples) produced a clear dissociation between loss and behaviour. Validation loss halved relative to the SFT rung (0.0065 terminal), yet development accuracy moved only from 81.2% to 81.5% on Qwen and from 83.1% to 82.7% on Llama. Under the flat aggregate, capability was *reshuffled* toward the self-harvest’s emphasis. Qwen’s RSFT rung gained four bridge_join questions but collapsed its factoid bucket from 18/20 to 7/20 through over-abstention. The collapse is a policy-specific data bias, not a property of the method: Llama’s own self-harvest induced no such artifact (its factoid bucket held at 16–17/20), and the subsequent DPO rung restored Qwen’s factoid bucket to 18/20. Self-harvested data inherits the quirks of the policy that generated it, a caution that recurs in the preference-learning analysis of Chapter 7.

44 Preference optimisation on merged pairs

The DPO rung merges the two rounds of attempt-protocol pairs into a single preference pool: 390 off-policy pairs from the teacher harvest plus 689 on-policy pairs from the Qwen self-harvest, 1,079 pairs in total. The pool includes the oracle-gated wrong-answer-repair pairs, in which the chosen plan is the repaired one and the rejected plan is its verifier-passing-but-wrong predecessor. Training continues the RSFT adapter for a single epoch (68 steps) at 5×10^{-6} with $\beta = 0.1$.

On the development set, DPO lifts Qwen from 81.5% to **83.5%**, recovering the factoid bucket and making it the Qwen ladder’s champion rung. On Llama the same procedure *regresses* the ladder from 82.7% to 77.3%. This asymmetry is not noise. Chapter 7 analyses the mechanism: near-miss on-policy negatives on the bridge bucket are string-adjacent to correct plans, and suppressing them displaces the correct modes; the pathology arrives at higher dose on Llama. Two remediation arms (a rebalanced sigmoid pool with a chosen-likelihood anchor, and an IPO variant) were trained within the pre-committed three-round development budget and did not beat the original rung (80.0% and 82.3% respectively). Both are reported as mechanism-explained negative results, and Llama’s selected configuration for everything downstream is its ladder-best SFT rung.

45 Step-1 distillation: the fully-local stack

Every system so far is a *hybrid*: local Step-2 planning under a cloud Step-1 briefing. The final form of the learning layer removes this last cloud dependency by distilling Step-1 itself. The filter here must be different, because a briefing is free-form text on the unverifiable side of the projection boundary, and no deterministic check can accept it directly. Instead, the training set consists of the 5,091 briefings whose *entire downstream pipeline outcome* was verified and oracle-correct. A briefing enters the pool only if the plan built on top of it executed, passed the verifier chain, and matched the oracle. This is the partial-verifiability filter applied one stage upstream; the verifiable end of the pipeline vouches for the unverifiable middle. Step-1 adapters were trained on both bases under the identical QLoRA recipe and paired with each base’s champion Step-2 rung (Qwen: DPO; Llama: SFT). The two adapters were served side by side on a single vLLM instance, with no cloud call anywhere in the stack.

On the development set the fully-local stacks score **86.2%** (Qwen, +2.7 over its hybrid) and **84.2%** (Llama, +1.1). The cross-base replication points to a recipe-level effect, and it is the third independent validation of the partial-verifiability claim, after teacher filtering and self-harvest. However, the development pairings are individually non-significant (+13/−6, McNemar $p = 0.167$ for Qwen; +14/−11, $p = 0.690$ for Llama), so this chapter draws no superiority conclusion from them. The fully-local-beats-hybrid claim is licensed only by the final_test pairings of Section 6.8, where it is tested at nine times the sample size.

46 The development ladder matrix

Table 6.2 assembles the full evaluation matrix on the development set under pipeline v2.2 and the uniform evaluation protocol. Its role is model selection and robustness analysis; none of its cells is a headline claim.

****Table 6.2 — Development ladder (compare_set_v4, $n = 260$, pipeline v2.2, evaluation protocol: repair budget 1, two plan samples, guided JSON on every rung). Artifacts under `outputs/eval/matrix_v2/`.****

Three readings matter. First, the scaffolding floor. The zero-shot rung — an untrained base model inside the full deterministic scaffold of grounding, compilation, exhaustive execution, verification, and guided decoding — already roughly doubles both RAG baselines (under the v1 pipeline, zero-shot Qwen scored 61.5% against 31.5% and 28.8%). Training gains are therefore added on top of an already-raised floor, and Chapter 7 decomposes the two contributions. Second, the training gain. SFT adds around 11 points to Qwen and 23 to Llama over their zero-shot rungs; RSFT is aggregate-neutral but redistributes capability; preference optimisation adds 2 points on one base and subtracts 5 on the other. Third, robustness to teacher nondeterminism. Because the cloud teacher is a live, nondeterministic service, we evaluated it three times, which defines a noise floor of $72.6\% \pm 1.1$. The Qwen DPO rung beats each of the three replicates individually (discordants +35/−5, +34/−4, +30/−5; McNemar $p = 3 \times 10^{-7}$, 5×10^{-7} , 2×10^{-5}), so the development-set

Table 6: Development-set ladder (compare_set_v4, pipeline v2.2): both bases, all rungs, with the teacher noise floor.

System	Qwen3-8B	Llama-3.1-8B
Zero-shot (full scaffolding, untrained)	70.4% (183/260)	60.0% (156/260)
SFT	81.2% (211/260)	83.1% (216/260)
RSFT	81.5% (212/260)	82.7% (215/260)
DPO	83.5% (217/260)	77.3% (201/260)
Fully-local (Step-1 + champion Step-2)	86.2% (224/260)	84.2% (219/260)
Teacher, 3 replicates	mean 72.6% $\pm$ 1.1 (71.9, 71.9, 73.9)	—
RAG baseline, naive / strong (v1)	31.5% (82/260) / 28.8% (75/260)	—

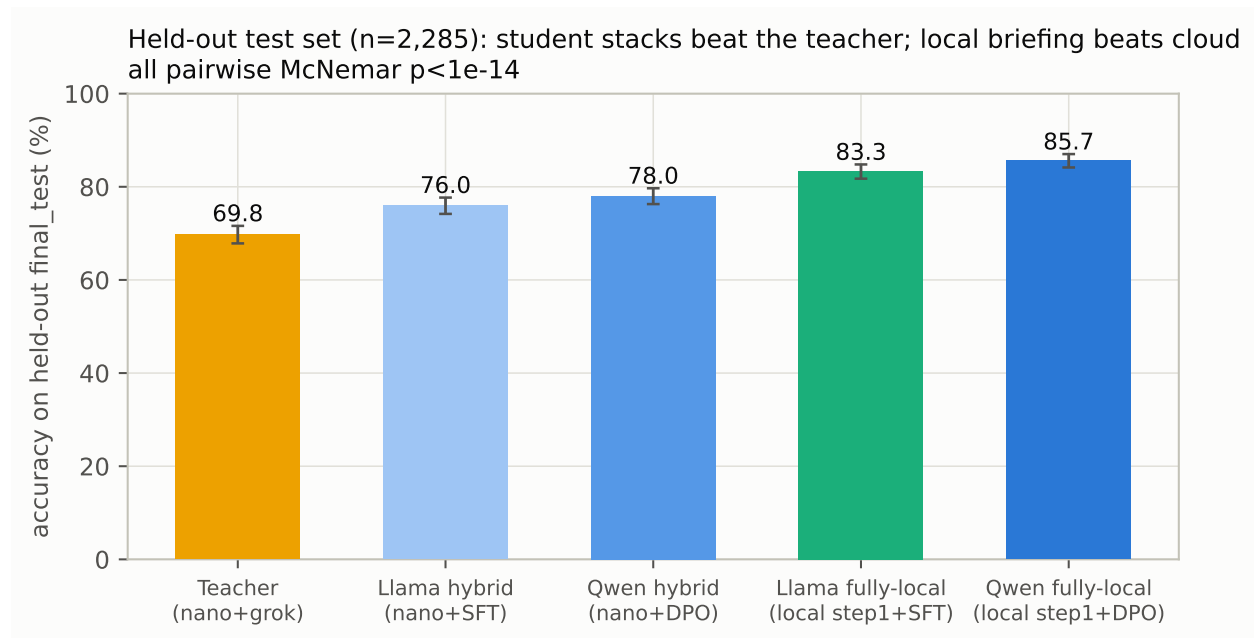


Figure 3: Held-out test set (n=2,285): the five-system scoreboard. Student stacks beat the teacher; the local briefing beats the cloud briefing. All pairwise McNemar $p < 10^{-14}$.

student-teacher gap cannot be an artifact of catching the teacher on a bad run. This three-replicate robustness role is exactly what the development set contributes to the student-versus-teacher claim; the claim itself is settled on final_test.

47 The confirmatory scoreboard on final_test

Each base’s hybrid champion and fully-local stack, together with a single teacher replicate, was evaluated exactly once on the frozen final_test set (n = 2,285, benchmark v4.1, evaluation protocol). All model selection preceded these runs; nothing was tuned afterwards.

Table 6.3 — Confirmatory five-system scoreboard (final_test, n = 2,285, v4.1, repair budget 1). All ten pairwise comparisons significant at McNemar $p < 10^{-14}$. Confidence intervals are 95% intervals on the paired accuracy difference. Artifacts under outputs/eval/final_test/.

|—|—|—|—|—| Teacher (single replicate) | 69.76% (1,594/2,285) | — | — | — | | Llama hybrid (nano Step-1 + SFT) | 75.97% | +6.21 [+4.68, +7.74] | — | -7.1 pt | | Qwen hybrid (nano Step-1 + DPO) | 78.03% (1,783/2,285) | +8.27 [+6.70, +9.85] (+263/-74) | — | -5.5 pt | | Llama fully-local (Step-1 + SFT) | 83.33% | +13.57 [+11.81, +15.32] | +7.35 [+5.73, +8.97] | -0.9 pt |

Table 7: Confirmatory five-system scoreboard on the held-out final_test (n=2,285); all pairwise McNemar $p < 10^{-14}$.

System	Accuracy	Δ vs teacher [95% CI]	Δ vs own hybrid [95% CI]	dev $\rightarrow$ test decay
--------	----------	------------------------------	---------------------------------	------------------------------

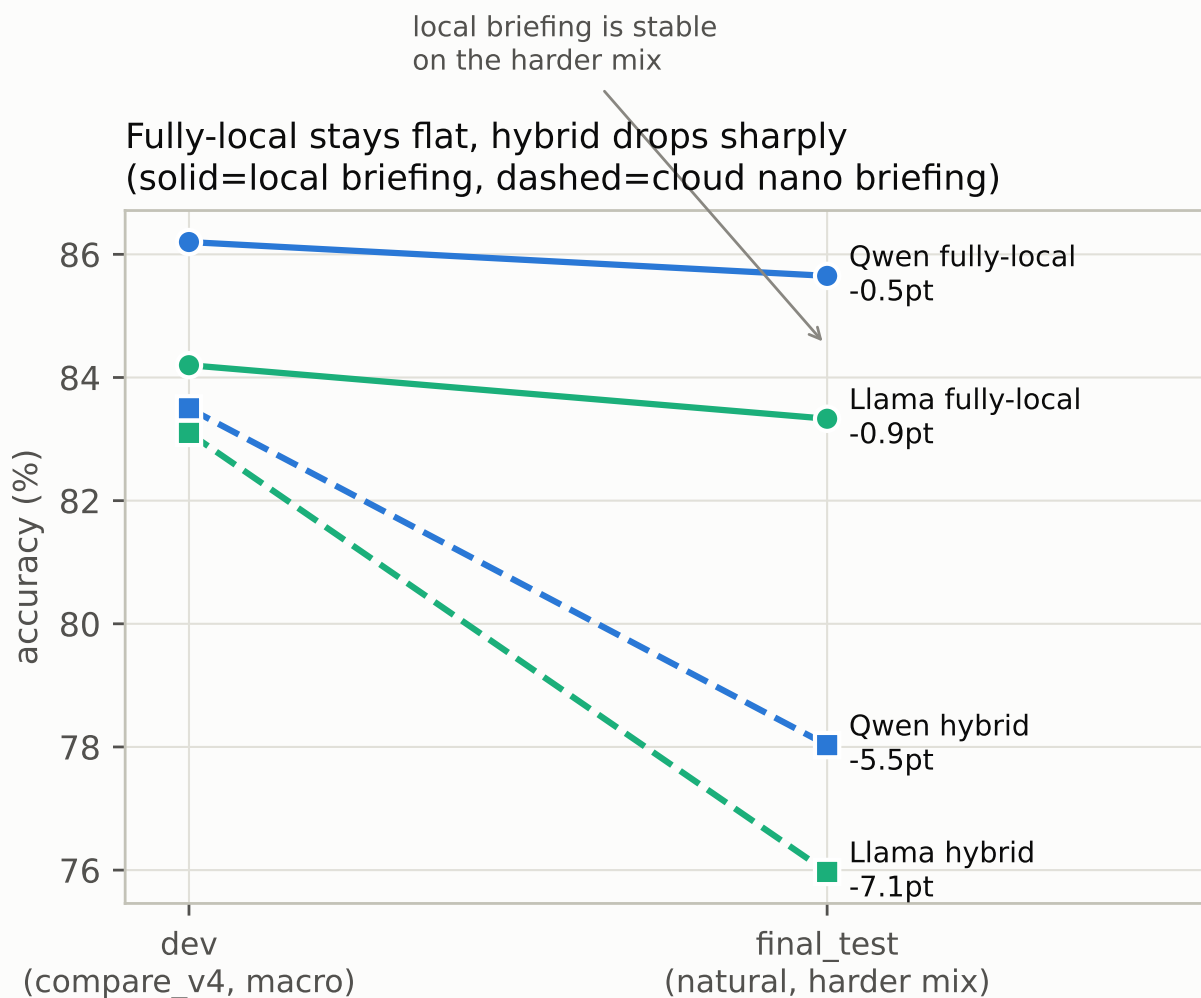


Figure 4: The four-point dev-to-test decay pattern: fully-local systems stay flat on both bases ($-0.5 / -0.9$ points) while hybrid systems drop sharply ($-5.5 / -7.1$). Solid lines: local briefing; dashed: cloud briefing.

The scoreboard delivers the chapter’s three claims at scale. First, every student system beats the teacher: even the weaker base’s hybrid clears it by six points, and the Qwen fully-local stack answers 406 questions the teacher misses while losing only 43 in the other direction. Per the pre-committed wording discipline, the student-versus-teacher comparisons in Table 6.3 are pairings against a *single* teacher replicate. The robustness of the student–teacher gap to provider nondeterminism is established by the three-replicate analysis on the development set (Section 6.7), and the two statements are kept deliberately separate. Second, on both bases the fully-local system beats its own cloud-hybrid counterpart by a nearly identical margin ($+7.35$ and $+7.61$ points). The claim that the development pairings could only suggest is confirmed here with $p < 10^{-14}$ on both bases. Third, the effect of the distilled briefing stage is strong enough that the weaker base with local briefings outranks the stronger base with cloud briefings (Llama fully-local 83.33% vs Qwen hybrid

Table 8: Bootstrap round 3 (r2): harvest yields keep rising while the evaluation gain saturates; reported as a supplementary row per the pre-committed gate.

Qwen fully-local (Step-1 + DPO)	85.65% (1,957/2,285)	+15.89 [+14.07, +17.70] (+406/-43)	+7.61 [+6.10, +9.13] (-
---------------------------------	-----------------------------	--	-------------------------

78.03%). This is a post-hoc, non-preregistered observation; we note it here and take it up mechanistically in Chapter 7.

The final column contains the most diagnostic pattern in the table. Moving from the development set to the harder natural distribution of `final_test`, both hybrids decay steeply (-5.5 and -7.1 points) while both fully-local systems are essentially flat (-0.5 and -0.9). The pattern holds in all four cells, across bases and configurations, at $n = 2,285$. It is the strongest single piece of evidence that Step-1 distillation acts as *denoising*: the local briefing stage, trained only on briefings whose downstream outcome was verified correct, stays in-distribution for the planner exactly where the cloud briefings become irregular. Chapter 7 gives the mechanistic decomposition of this asymmetry.

Finally, the disclosure required by Chapter 4: nineteen of the 2,285 test questions string-match the `_UNSUPPORTED_CUES` list that was extended during development. The dual report excluding these rows moves every headline by at most 0.21 points (teacher 69.76 $\rightarrow$ 69.55; Qwen fully-local 85.65 $\rightarrow$ 85.57) and changes no ranking and no significance verdict. The deltas are uniformly *negative* because the cue rows were mostly answered correctly by all systems. Removing easy rows lowers every accuracy slightly; the deltas do not indicate any system-specific advantage.

48 Round-2 self-harvest: bootstrap convergence

Does the loop keep paying? We ran a second self-harvest round (r2) under the fully-local champion configuration: local Step-1 adapter and the DPO champion as Step-2, temperature 0.7×4 , with the data engine now containing no cloud model at all. Its outcome was mapped onto gates pre-committed *before any r2 number existed*. The gates were: a headline swap only if r2 beat the champion by at least 3 development points with a significant bridge improvement; a matrix row if it was at least neutral; and an honest diminishing-returns report otherwise.

At the harvest level the loop is still climbing steeply. Answerable yield reached **86.3%** (teacher 65.5% $\rightarrow$ r1 76.6% $\rightarrow$ r2 86.3%), correct abstention 97.6%, and the bridge_join bucket — the task’s hardest family — jumped from 52% oracle-correct in r1 to **75%** in r2. The DPO policy samples correct bridge plans far more often than the SFT policy did. At the training level, however, the gain has stopped. The r2 adapter (iterative RSFT continuing the DPO champion on its own harvest) scored 86.5% on the development set against the champion’s 86.2%: +0.4 points, discordants +3/-2, McNemar $p = 1.0$. Its bridge bucket improved by exactly one question (15/20 vs 14/20, $p = 1.0$). Per the pre-committed gates this is a supplementary-row outcome: the headline system remains the fully-local DPO stack, and r2 is reported as a data point on a convergence curve rather than as a new rung.

The convergence curve is itself a finding. Round 1 converted an 11-point yield gain into a significant ladder gain; round 2 converted a further 10-point yield gain into statistical noise. The verifier-gated self-improvement loop has a natural ceiling on this task. Once the policy already solves, under sampling and repair, nearly everything the verifiable region can certify, additional verified data re-teaches what the policy already knows. Yield keeps rising precisely because the policy improved; training gain stalls precisely because the remaining errors live where the verifier cannot see. The Llama arm of r2 was not trained. The convergence claim is established on the main base; single-base evidence is enough for a convergence observation (unlike the DPO asymmetry, no cross-base contrast is claimed), and the remaining compute budget went to the pre-registered compositional probe.

49 Cost analysis

The cost asymmetry between the teacher and its replacements is a result in its own right. The teacher harvest is a metered, latency-bound process. At roughly 13 questions per minute it took about twelve hours of wall-clock over the 9,267-question pool and consumed on the order of tens of thousands of billable API calls (each question costs one Step-1 call, one to two Step-2 calls, and up to two repair round-trips). The self-harvest replaced the metered Step-2 with a local vLLM endpoint and ran at 85 questions per minute — the full pool in under two hours, at zero marginal cost for every sampled candidate. This is what makes the temperature-0.7 $\times$ 4 rejection-sampling regime economically trivial for a student and prohibitive for a metered teacher. By r2 the engine contained no cloud model at all.

Training costs are one-off and modest. Each SFT-class rung is roughly three hours on a single H100 (2h43m for Qwen SFT, 2h22m for Llama SFT, 2h42m for RSFT), the DPO rung thirteen minutes, and no stage requires more than one GPU. On the inference side, the fully-local champion serves both stages as two LoRA adapters on one vLLM instance on one GPU; the entire 2,285-question `final_test` evaluation of the headline system ran without a single external API call. The deployment case for the fully-local stack is therefore not accuracy traded for cost but accuracy *and* cost. It is 15.89 points more accurate than the cloud teacher it learned from, at zero marginal inference cost, and with no procurement data leaving the machine.

50 Summary

Three instantiations of the same principle produced three compounding results. Teacher- filtered distillation turned a 65.5%-yield data engine into students that clear the teacher’s own evaluation accuracy. Verifier-gated self-harvest then out-yielded the teacher’s engine by 11–13 points and converted its blind-region failures into preference-learning fuel. Step-1 distillation through the whole-pipeline filter removed the final cloud dependency and, on the held-out confirmatory set, produced the ordering teacher < hybrid < fully-local on both bases, with every pairing significant at $p < 10^{-14}$ and near-zero dev-to-test decay. A second self-harvest round located the ceiling of the loop: harvest yield keeps rising while training gain stops, which fixes the boundary of what verifier-gated bootstrapping can extract from this task. Supervision never left the verifiable region; the systems it trained did. Chapter 7 turns from the scoreboard to the mechanisms behind it.

51 Analysis: Mechanisms, Noise Floors, Failure Modes, and Ceilings

Chapter 6 reported the scoreboard; this chapter explains the mechanisms behind it. The aim is that the reader believes the results not because the deltas are large, but because the failure structure beneath them is accounted for. Seven analyses carry that burden. Section 7.1 gives a mechanism-explained negative result for preference optimisation. Section 7.2 traces the four-point dev-to-test decay pattern, which identifies distillation-as-denoising as the operative mechanism, and adds a post-hoc cross pairing that isolates briefing quality as the dominant variable. Section 7.3 measures the teacher provider-noise floor that disciplines every teacher-relative claim in this dissertation. Section 7.4 decomposes the error modes and identifies a structural ceiling the students share with their teacher. Section 7.5 reports a schema-format diagnostic of what fine-tuning actually teaches, together with its own instrumentation history as a methodological finding. Section 7.6 examines the hard-composite slice on which the student’s advantage is largest. Section 7.7 discloses the pilot results of a pre-registered compositional probe, exactly as far as they reached.

52 DPO and the toxicity of near-miss negatives

Why did DPO help on one base and hurt on the other? At the close of Chapter 6, Direct Preference Optimisation was the champion rung of the Qwen ladder: DPO-v1 reached 83.5% on the development set (217/260), a net +2.0 points over the RSFT rung beneath it (81.5%). That aggregate hides a regression which the per-bucket decomposition exposes at once. On the hardest question family, `bridge_join`, the RSFT rung answered 16 of 20 correctly and the DPO rung only 11. The preference stage gained in aggregate while

losing on exactly the capability the preference pairs were richest in. On the second base the same treatment was not just uneven but net-destructive: Llama DPO-v1 scored 77.3% against its RSFT parent’s 82.7%, a −5.4 point regression. Read together, the two bases trace a dose-response curve for a single pathology — mild on Qwen (+2.0 net, with a localised bridge regression) and severe on Llama (−5.4 net) — rather than two unrelated tuning accidents.

The training internals of the Llama run identify the pathology. Its DPO loss saturated at 0.03 with pair-classification accuracy of 99.4%. The log-probability of the chosen responses sat at −7.7, essentially memorised from the RSFT stage that preceded it. The growth in preference margins came entirely from pushing the log-probability of *rejected* responses down, from −185 to −203. Nothing was being learned about the chosen plans; they were already at ceiling. The optimiser therefore satisfied the objective by displacing probability mass away from the rejected plans and, as we argue below, away from correct plans adjacent to them. The damage had a behavioural signature visible before any accuracy number was computed: during its evaluation the model’s generation throughput collapsed by roughly a factor of nine, the symptom of output distributions flattened under guided decoding.

The natural response is to blame the recipe, and the recipe was genuinely open to criticism. The v1 pair pool had never been bucket-capped: 71% of its pairs came from the bridge and set families, and 766 of its 1,079 pairs originated from oracle-gated repair traces. The loss also carried no anchor on the chosen responses (no auxiliary likelihood term). We therefore ran a two-arm recipe-fix ablation following contemporary curation practice. The pools were rebuilt on-policy-first (for Qwen, 389 pairs of which 94% were on-policy), bucket-capped, and deduplicated to a balanced composition. Arm A used the sigmoid loss with an auxiliary chosen-likelihood term; arm B replaced the objective with IPO. Both trained for three epochs after a one-epoch pilot showed initial margins of −17: on-policy pairs are hard against a base-model reference, and the first run had simply been undertrained at 25 steps.

Both arms failed. Arm A scored 80.0% with a bridge sub-score of 5/20; arm B scored 82.3% with bridge 9/20. Both lose to the unfixed champion DPO-v1 (83.5%, bridge 11/20), and all three lose the bridge bucket to plain RSFT (16/20). The recipe fixes did not just fail to help. The *cleaner*, more on-policy, better-balanced pools produced *worse* bridge performance than the naive pool they replaced.

That inversion is the diagnostic fact, and it points to a mechanism rather than a tuning error. The rejected bridge plans in an on-policy pool are *near-miss* negatives. They are string-adjacent to correct bridge plans: the same four-variable dataflow, differing only in the return target or a single slot. When the objective suppresses them, it also drags down the probability of the correct modes next to them. An anchor term or the IPO objective cannot protect what it cannot separate, because the protected and suppressed manifolds coincide. This is consistent with the likelihood-displacement analysis of Razin et al., in which preference optimisation is most destructive exactly when chosen and rejected responses are similar. On this reading, balancing and on-policy filtering *concentrated* the poison: they raised the fraction of pairs whose negative sat closest to the correct answer. The dose-response across bases also follows naturally. The Llama policy, whose RSFT stage had already memorised the chosen set, had nothing left to gain and everything to displace.

Two consequences were adopted, and both are visible elsewhere in this dissertation. First, the claim is bounded. These experiments do not show that DPO is toxic in general; they show it is hazardous in *near-miss-negative regimes*, where the verifier’s rejects are structurally adjacent to its passes — which is exactly the regime a verifier-gated pipeline produces. Second, the bridge-capability lever was moved from stronger suppression to *additive* learning: the r2 bootstrap round (Chapter 6) consolidates correct bridge plans by RSFT-style self-distillation from the DPO policy, with no negative term at all. Within the terms of Pillar 3, the finding is a controlled contrast between subtractive and additive use of the same verifier signal. Filtering what enters the positives is safe; suppressing what the filter rejects is not, whenever the rejects live next door to the truth.

53 The four-point decay pattern: distillation as denoising

Why do the hybrid systems decay from the development set to the test set while the fully-local systems stay flat? The most consequential single pattern in the confirmatory results is not any one accuracy but

the *shape* of this decay across the four student systems. Both fully-local systems are essentially flat from the development set to the held-out test set: Qwen fully-local decays -0.5 points ($86.2\% \rightarrow 85.65\%$) and Llama fully-local -0.9 points ($84.2\% \rightarrow 83.33\%$). Both hybrid systems fall steeply on the identical transition: Qwen hybrid -5.5 points ($83.5\% \rightarrow 78.03\%$) and Llama hybrid -7.1 points ($83.1\% \rightarrow 75.97\%$). The pattern is four-for-four and cross-base: flat wherever the understanding stage (Step 1) is the distilled local adapter, steep wherever it is the cloud model, with the *same* planning checkpoints on each base. Whatever explains the decay must therefore live in the briefing stage, not in the planner and not in the base model.

Three strands of evidence, each with its own experiment, converge on one account.^[^strands] First, composition. The final test set’s natural question mix upweights the bridge and comparison families to 28.7% of items, against 15.4% under the development set’s uniform bucketing. These are exactly the families where the structure and regularity of the intermediate briefing most strongly determine whether the planner can produce a correct multi-variable plan. Second, distributional match. The cloud model’s briefings on hard questions are *irregular* — free-form in structure and emphasis — whereas the student’s Step-2 planner was trained on the regular briefing distribution. The distilled Step-1’s output lands inside the distribution the planner knows; the cloud model’s improvisations land outside it. Third, boundary behaviour. An autopsy of the hybrid system’s answerable-to-null down-flips on the test set found that four of six were *deterministically* reproducible conservative abstentions (boundary questions pushed to the abstain side by irregular briefings), with only two of six attributable to provider churn. The three strands are one mechanism seen at three grains: the harder mix concentrates weight on briefing-sensitive questions, irregular briefings fall outside the planner’s trained distribution there, and at the decision boundary that mismatch resolves into abstention. Distillation of the understanding stage acts as *denoising*: the Step-1 student, trained only on briefings whose entire downstream pipeline outcome was verified and oracle-correct, reproduces the teacher’s competence stripped of its irregularity.^[^fileorder]

^[^strands]: Strand one is a distributional census of `final_test` versus the development buckets; strand two rests on the Step-1 distillation filter described in Chapter 6 (5,091 briefings retained solely on whole-pipeline verified-and-oracle-correct outcome); strand three is the down-flip autopsy of 2026-07-07, in which the six answerable→null flips were re-run through a live hybrid configuration: 4/6 reproducibly null, 2/6 flipped back correct.

^[^fileorder]: A rolling-window audit initially suggested a mid-run incident: both the hybrid system and the teacher show their worst accuracy window near file position 850. The audit resolved this benignly. Positions 800–1000 of `final_test` are a 200/200 cluster of `bridge_join` items in file order, so the co-located dip is that cluster, not a temporal API event; the zero-cloud fully-local run swings at the same positions.

One further observation sharpens the conclusion, and it must be read with its label attached: it is a post-hoc pairing, not a pre-registered comparison.^[^posthoc] The weaker base with the local briefing beats the stronger base with the cloud briefing. Llama fully-local scores 83.33% against Qwen hybrid’s 78.03% (discordant pairs $+252/-131$, $\Delta = +5.30$ points, 95% CI $[+3.62, +6.97]$, $p = 8.7 \times 10^{-10}$), even though Qwen sits 2–3 points above Llama in every same-briefing configuration in the matrix. Briefing quality is the dominant variable: the quality difference between the two Step-1 stages outweighs the capability difference between the two base models.

^[^posthoc]: This cross pairing was noticed after the five-pairing matrix completed and was not pre-registered; it is reported as a mechanism observation only and carries no headline weight.

54 The teacher noise floor

How much does the teacher differ from itself? Every claim in this dissertation of the form "system X differs from the teacher by Δ " presupposes an answer to that question. Three identical v2.2 evaluations of the teacher pipeline on the development set — same questions, same configuration, different nights — returned 71.9%, 71.9%, and 73.9%: a mean of 72.6% with a standard deviation of 1.1 points. The stability of the totals is deceptive. Pairwise, the three replicates disagree on between 13 and 22 individual questions. The provider’s unseeded temperature-zero decoding churns roughly 5–8% of predictions between runs that are configured identically, and the churn largely cancels in the aggregate.

Two disciplines follow, and both are enforced throughout. First, no single-run teacher delta below roughly three points is claimable. Teacher-relative headline claims either clear the floor by a wide margin (the student–teacher gaps of Chapter 6 exceed it several times over) or are paired against *each* replicate separately. Second, an honest account of where we were caught by this ourselves. During the v2.0→v2.1 pipeline iteration we chased an apparent teacher regression (70.0% → 66.5% → 66.1%) as if it were entirely a scaffolding defect. The later adversarial review established that the chase was partly noise-chasing. The v2.1 gate-narrowing was a real fix, with deterministic answerable-to-null signatures behind it, but the residual 66.1% was a bad draw from the noise floor, not evidence of remaining gate damage. We record the episode as a lesson because it is the mirror image of the discipline: attributing a within-floor fluctuation to a cause is as much an error as claiming one.

The floor also disposes of a tempting misreading of the confirmatory set. The teacher’s 69.76% on `final_test`, below its 72.6% development mean, is difficulty, not API degradation. An integrity audit compared the teacher’s predictions on the 260 development questions embedded within `final_test` across the two runs and found the teacher *net positive* (+6) on the later run. The run that produced 69.76% was, if anything, a slightly favourable draw; the lower total reflects the harder natural mix of the full test set.

55 Error-mode decomposition and the boolean structural ceiling

Which errors does scaffolding fix, which does training fix, and what remains when both levers are exhausted? The v1 evaluation matrix supplied the error taxonomy from which the v2 pipeline was built. Of the champion rung’s development-set errors, 21 were abstentions on answerable questions, 12 were answers on unsupported questions, and 22 were wrong values. Chapter 5’s worked example traced these to four precise scaffolding defects: an over-strict type gate on empty-value lookups, an over-permissive alias substring rule, a consistency-check bypass on the compiled fast path, and a verifier-invisible count-of-intermediate-set misread. Each was fixed deterministically and regression-tested against real pipeline intermediates. The decomposition’s purpose in this chapter is the division of labour it establishes for Pillar 2. The first two error modes are dominated by *scaffolding*: they moved when the checks moved, for every system including the untrained zero-shot rung. The wrong-value mode is dominated by *planning quality* and moved mainly with training. What scaffolding fixes, it fixes for all systems at once (the v2.2 zero-shot floor rose to 70.4% from v1’s 61.5%); what training fixes, it fixes per rung, on top of whatever floor the scaffolding provides.

What remains when both levers are exhausted is visible in one bucket with unusual clarity. On the development set’s boolean family (n=20), nearly every system in the matrix lands on 14/20: the fully-local Qwen champion, the fully-local Llama system, and the hybrid champion alike (Llama-SFT and one teacher replicate land on 13/20). More telling than the matching totals is the matching items. The *same six questions* are missed by the fully-local Qwen system, the fully-local Llama system, and the hybrid champion, six of six overlapping. All six are also missed by every one of the three teacher replicates.^[^booleansrc] This is not a training gap. No rung of either ladder, at any training dose, touches these questions, and neither does the teacher under any draw. It is a shared structural ceiling: some property these six questions require is not currently expressible through the pipeline’s plan language or its checks, for teacher and students alike. Distillation cannot exceed what neither the teacher nor the scaffolding can express. The ceiling is pipeline-bound, and raising it is scaffolding work (moving the projection boundary), not learning work. We state this as a shared expressiveness ceiling and no more. The evidence does not distinguish which component of the chain (plan vocabulary, grounding, or the questions’ own construction) carries the limitation, only that it is common to every system evaluated.

[^booleansrc]: Computed from the per-item results files of the v2.2 matrix (outputs/eval/matrix_v2/*/compare_cicada.results.jsonl).

56 The schema-format diagnostic: what does fine-tuning actually teach?

Does fine-tuning teach planning, or only the output format? The training ladder of Chapter 6 measures accuracy under guided decoding, which enforces the plan schema at generation time. This invites a deflationary reading: perhaps fine-tuning teaches nothing but the format, and the ladder’s gains are an artefact

of format acquisition. The schema diagnostic was designed to test this, with success criteria pre-committed before any output was read. Its result is a clean separation, but the path to that result matters as much as the result, and we report both.

The diagnostic evaluates three arms — the untrained Qwen3-8B base, the SFT rung, and the DPO rung — on 100 development questions with *no* guided decoding. Each output is scored on two metrics: *grounded-any-schema* (does the output contain a question-grounded plan in any schema at all?) and *target-contract-conformant* (does it match the exact contract the compiler consumes?). The result: grounded-any is 98/98/99 across base/SFT/DPO, but target-contract conformance is 3/98/99. Opening ten randomly chosen base-arm outputs (a requirement, not a courtesy; see below) confirmed what the numbers suggest. The untrained base emits question-grounded plans with *real* entities, such as Wessex Water, NHS England, and genuine CPV codes and years, organised into a coherent, self-invented schema (op:filter_records, args.filters, id/inputs) that the compiler cannot consume. The base can plan; it cannot emit the executable contract. Fine-tuning’s measured contribution is therefore *conformance to the specific executable contract* ($3 \rightarrow 98/99$), not planning ability itself. This is exactly the control-variable result the ladder needs. Guided decoding at evaluation time enforces the one thing that is near-free for trained models and near-impossible for the base, so the accuracy ladder isolates planning *quality within the contract* rather than the ability to emit it. Two honesty caveats attach. The diagnostic stored only failures, so trained-arm passes were not individually inspected; the claim rests on the base arm’s verified grounded-but-off-schema pattern. And one base abstention expressed through a return-operation field was under-counted as empty. The 3-versus-98 direction is insensitive to both.

The raw-inspection requirement exists because this diagnostic produced three consecutive wrong answers before it produced a right one, and each wrong answer was caught only by opening raw outputs. The first instrument *over-counted* base-arm parse failures: a greedy JSON extractor was confused by markdown fences. We patched it. The second *over-counted* base-arm successes: a well-formedness gate scored 100/100 for the base, and four spot-checked outputs revealed these to be identical degenerate shells, structurally present but semantically empty, so a content gate was added. The third *under-counted* the base: the content gate looked for filters only at the trained schema’s location, scored the base’s genuinely grounded plans as zero, and produced a suspiciously clean 0/94/99. We retracted that result before publication and replaced it with the dual-metric design above. Three gate bugs, three opposite-signed biases, each one capable of carrying a false conclusion into this dissertation. Each was caught by the same act: refusing to cite a gate-produced number without reading the raw outputs behind it. We generalised the episode into a standing rule for every gate-decided number in this work — ten deterministic-random raw outputs, passes and failures, human-read before the number enters the results table. We offer the audit trail itself as an evaluation-methodology contribution. Every gate is a measurement instrument, and an unverified instrument that produces a clean number is a liability precisely *because* the number is clean.

57 The hard-composite slice: where the student’s advantage is largest

Where is the student’s advantage over the teacher largest? Chapter 6 established that the fully-local student beats the teacher on aggregate; the slice analysis locates where. The development-era analysis partitions the evaluation set into an iid slice ($n=103$) and a hard-composite slice ($n=136$). The latter comprises the hard operator families, second-level paraphrase rewrites, and abstention classes. One naming discipline governs everything said about it: this is a *difficulty composite*, not a held-out distribution. The class is present in both training and test (with disjoint plans and novel surfaces), so no claim of out-of-distribution generalisation is made or licensed here; compositional novelty is the business of the pre-registered probe in §7.7.

The pattern, not any single cell, is the finding. Every teacher replicate decays by 6 to 12 points from the iid slice to the hard composite; the hybrid student decays by 7.7; the fully-local student shows no measurable decay (+0.8 points, $n=136$). The fully-local system uses the *same* planning checkpoint as the hybrid, so the difference is attributable to the distilled understanding stage. On the hard-composite slice itself the fully-local student’s advantage over the teacher reaches 18–20 points, its largest margin anywhere in the evaluation. The student trained only on verifier-passed supervision holds its accuracy exactly where the far larger teacher loses it. This is the mechanism evidence for the one-sentence thesis in its sharpest available form. The caveats are stated with the finding: slice sizes are modest (103 and 136), the slopes

Table 9: Accuracy on the iid versus hard-composite slices of the development set (absolute values; the pattern, not single points, carries the claim).

System	iid (n=103)	hard-composite (n=136)	slope
Teacher r1 / r2 / r3	75.7% / 73.8% / 78.6%	65.4% / 67.6% / 66.2%	-10.3 / -6.1 / -12.5
Qwen DPO-v1 (hybrid)	86.4%	78.7%	-7.7
Qwen fully-local	84.5%	85.3%	+0.8

carry corresponding uncertainty, and the wording "no measurable decay" describes this pattern at this n ; it is not a claim of proven invariance. The four-point dev-to-test pattern of §7.2 — the same flat-versus-steep signature at $n=2,285$ — is the larger-scale corroboration that keeps this small- n table from bearing weight alone.

58 The compositional probe: pilot findings and a boundary characterisation

Does the system generalise to plan compositions it has never seen? The claims above stop deliberately short of that question, which was assigned to a separate pre-registered instrument: `ood_probe_v1`. The probe has five question templates whose structural signatures — rebuilt from gold-plan flat specifications, not template names — are absent from the entire corpus: three bridge compositions (B1 `bridge_argmax`, B2 `bridge_top_k`, B3 `bridge_compare`) and two non-bridge compositions (N1 grouped temporal `argmax`, N2 per-side filtered `sum-compare`), with three-branch success criteria committed before generation. The probe was *paused at the pilot stage* by an explicit scope decision before full generation began. No part of the planned ~600-row evaluation was run, and nothing in this dissertation cites it as a generalisation result. The pilot itself, however, produced findings worth reporting exactly as far as they go.

First, the novelty gate passed its self-audit 10/10 in both directions: five existing question families were correctly recognised as present in the corpus signature inventory, and all five probe templates as absent. Second, the pilot corrected the pre-registration itself. N2’s claimed novelty on the comparison-side-type axis proved false on faithful reconstruction: its comparison side is an existing type, and the template is novel only on the filter-slot axis, which makes it materially the weakest of the five. The correction is recorded against the pre-registration rather than papered over. (The reconstruction also revised an ad-hoc signature census from 50 to 38 under the committed five-field definition; novelty assertions are unaffected under both granularities.)

Third, and scientifically the most interesting, the executability pilot ran five hand-written probe questions through the exact production entry point of the fully-local pipeline. That mandate has a history: an earlier hand-wired bypass of the production path had produced a nearly *inverted* executability table and was voided. The lesson, of a piece with §7.5, is that a pilot that does not enter through the front door measures a different system. Through the production path, all three bridge compositions executed end-to-end, including the highest-risk B3, in which a comparison decomposes into two bridged sub-plans. Both non-bridge compositions failed, for genuine and informative compile-time reasons. The executor’s `argmax` returns an extremum record and offers no grouped-`argmax`, so N1 is rejected at intent compilation. The two-sided sum-comparison of N2 drops the per-side year filters that distinguish its arms. The system’s compositional coverage is therefore *asymmetric*. Novel compositions within the bridge family — the family the training signal was richest in — execute, while the two non-bridge aggregation compositions chosen for the probe fall outside the executor’s current expressible plan space. We report this as a boundary characterisation of the executor, not as a generalisation result in either direction. It says where a full probe could measure generalisation, and where the pipeline itself, not the learner, is the binding constraint: a compositional cousin of the structural ceiling of §7.4.

59 Synthesis

The analyses compose into a single picture. The verifier-gated pipeline creates an unusual learning environment. Its positives are clean, which is why additive distillation keeps working (§7.2, §7.6). Its negatives are near-misses, which is why subtractive preference learning is hazardous (§7.1). Its teacher is noisy in a measurable, disciplinable way (§7.3). Its residual failures split cleanly into what scaffolding fixes, what training fixes, and what neither currently can (§7.4, §7.7). The diagnostic apparatus that produced these conclusions required the same scepticism as the systems it measured (§7.5). On this account, the headline numbers of Chapter 6 stand not as fortunate draws but as the visible surface of a mechanism — denoised supervision from the verifiable region, learned competence extending past it. The failure modes of that mechanism have been located, measured, and where possible explained. Chapter 8 states the conclusions this evidence supports and the limits it does not cross.

60 Conclusion, Limitations, and Future Work

61 Summary of the dissertation

Chapter 7 explained the mechanisms behind the results. This chapter states what the dissertation has shown, where its evidence stops, and what should come next.

The dissertation set out to defend a single sentence: **supervision flows only from the verifiable region, yet the learned competence extends beyond it**. Every training example used anywhere in this work passed a deterministic compile–ground–execute–verify chain. During training-data harvest only, it also passed an agreement check against an independently computed oracle — the oracle filtering candidates, never authoring them. Nothing the checks could not vouch for was learned from. Yet the systems trained this way did not stop at the projection line. The fully-local student holds its accuracy on precisely the hard-composite slice where its cloud teacher decays by 6–12 points, and it stays flat across the dev-to-test transition on which every cloud-briefed configuration falls steeply.

The system that carries this argument, CICADA, is built in three layers. A *hard-verification layer* projects every property of the question-to-answer chain that can be projected onto structure or execution, and pins it with a deterministic check; the checks shrink the blind region layer by layer without ever removing it. An *abstention layer* meets the unprojectable residue — the purely semantic correspondence between plan and question — with calibrated refusal, tested by three families of abstention traps built into the benchmark as first-class citizens. A *learning layer* draws supervision only from what the first layer passes, and the competence it learns reaches back over part of what the first layer cannot see. The empirical summary is the five-system scoreboard on the held-out test set ($n=2,285$): teacher 69.76% < Llama hybrid 75.97% < Qwen hybrid 78.03% < Llama fully-local 83.33% < Qwen fully-local 85.65%, with every pairing significant at McNemar $p < 10^{-14}$. The fully-local champion exceeds its cloud teacher by +15.89 points (95% CI [+14.07, +17.70]) and its own cloud-hybrid counterpart by +7.61 points (CI [+6.10, +9.13]), on a single local GPU at zero marginal API cost.

The four contributions announced in Chapter 1 were delivered as follows.

Pillar 1, the bootstrap closed loop, was instantiated three times. First, filtering the untrusted teacher’s outputs into clean supervision (harvest yield 65.5% oracle-correct on answerable questions). Second, a self-harvest round in which the student’s own filtered generations exceeded the teacher’s yield (76.6% on Qwen, 78.1% on Llama; these are harvest yields, a data-engine metric distinct from system accuracy). Third, the denoising distillation of the understanding stage that produced the fully-local stack.

Pillar 2, gain decomposition, separated the hard-verification layer’s contribution from the learning layer’s. The untrained zero-shot rung with full scaffolding reaches 70.4% on the development set — the scaffolding floor — and the training ladder adds its gains on top of that floor. The v1→v2.2 pipeline iteration showed that the projection boundary itself moves under engineering effort.

Pillar 3, DPO near-miss toxicity, is a mechanism-explained negative result with a cross-base dose curve. Preference suppression of verifier-rejected near-miss negatives degraded exactly the capability its pairs were richest in, mildly on one base and severely on the other, and a two-arm recipe-fix ablation failed in the direction the mechanism predicts.

Pillar 4, benchmark methodology, produced the 12,828-question benchmark with dual-implementation oracles at 99.88% measured agreement and plan-level split isolation. Its development/confirmatory discipline survived an adversarial review — including that review’s findings against us, which were adopted and disclosed rather than argued away.

62 Limitations

A dissertation whose central instrument is a filter owes the reader an equally disciplined account of what its evidence does not cover. Each limitation below is stated with its factual bound: what the claim would need in order to widen, and how far the present evidence actually reaches. Where a concrete next experiment exists, it is named. None of these limitations was found by a reviewer first; each is carried forward from the point in Chapters 4–7 where it arose.

A single self-built benchmark in a single domain. All claims in this dissertation are claims about UK-procurement knowledge-graph question answering, on a benchmark we constructed ourselves. The architecture-level claims — the three-layer decomposition and the value of verifier-gated distillation — rest on within-domain ablations. No evidence here licenses their transfer to another schema or domain. The cross-schema probe of §8.3 is the named test.

The hard-composite slice is a difficulty composite, not a holdout. The slice on which the student’s advantage is largest (Chapter 7) is defined as hard operators $\cup$ second-level rewrites $\cup$ abstention classes. Both training and test contain this class, with disjoint plans and novel surfaces. It measures robustness to difficulty, not out-of-distribution generalisation, and we have not named it "OOD" anywhere in this work — the naming discipline is itself the limitation, honestly carried. The strict holdout the benchmark does contain is a single test-only template family of small n , reported here rather than in any headline.

Compositional generalisation is unproven. The pre-registered compositional probe (`ood_probe_v1`) was paused at its pilot by an explicit scope decision; the full ~600-row evaluation was never run. What exists is disclosed in §7.7: a 10/10 novelty-gate self-audit, an honest weakening of one template’s novelty claim, and a production-entry executability pilot showing asymmetric coverage (novel bridge compositions execute; the two chosen non-bridge compositions fail at compile time). No compositional claim is made on this evidence.

The teacher–student comparison is asymmetric. The teacher operates zero-shot in-domain while the students are fine-tuned on in-domain data. Student-beats-teacher results are therefore claims about the *pipeline-plus-distillation recipe* — that a bootstrapped local stack outperforms the deployed cloud configuration it was distilled from — not claims that an 8B model outranks its teacher in model quality. The symmetric comparison is exactly what the compositional probe was designed to provide, since on its templates both teacher and student are zero-shot. The design is frozen, the pilot is complete, and the full run is deferred, not abandoned.

The teacher is a single replicate on the final test set. Provider-noise robustness lives on the development set, where three identical teacher runs establish a noise floor of $72.6\% \pm 1.1$ and the student beats each replicate separately. The final-test teacher pairing is labelled single-replicate wherever it appears. The +15.89-point gap exceeds the measured floor many times over, but the floor is the reason we can say so.

Preference optimisation was unstable in this regime. The DPO stage’s gains are real but fragile: the same recipe that added +2.0 points on one base removed 5.4 on the other, and the recipe-level fixes failed. We report the champion configuration with its mechanism-level explanation, not as a recommended general recipe.

The bootstrap converges early. The second self-harvest round (r2) lifted harvest yields substantially (answerable 86.3%, bridge 52% $\rightarrow$ 75%) yet delivered +0.4 points on the development set — not significant,

and adjudicated to a supplementary row under the pre-committed gate. The verifier-gated loop has a natural ceiling on this task; the first round captured essentially all of the transferable signal. Bootstrap claims in this dissertation are one-round claims.

Development-derived cues touch 19 test rows. Five unsupported-question cue strings were derived from development-set errors, and 19 of 2,285 test rows string-match them. The dual report bounds the effect: excluding those rows moves every headline by at most 0.21 points, with rankings and significance unchanged (the deltas are negative because the matched rows were mostly answered correctly by all systems). Two verbatim train-test duplicate questions, found by the adversarial review, are likewise disclosed and excluded from final statistics.

Correctness is convention-relative, and surfaces are synthetic. The oracles encode documented interpretive conventions; the 99.88% dual-implementation agreement (14,752/14,770, residual 18) bounds but does not remove convention risk. The benchmark’s question surfaces are LLM-generated across six style axes — wider than template English, but not equal to real user language.

63 Future work

Reinforcement learning from the verifier as reward. The pipeline already contains what RLVR-style methods require: a dense, deterministic, non-learned reward source. Chapter 7’s negative result argues for the direction — additive, on-policy reinforcement (GRPO-style) rather than pairwise suppression of near-miss negatives — and the r2 convergence result says what a new method must beat. One caveat is owed in advance. The verifier’s blind region is exactly where reward hacking would live: a policy optimised against the verifier is optimised to *pass checks*, and plans that are check-passing but semantically wrong receive positive reward by construction. Any RLVR follow-up must therefore carry the abstention layer and oracle-agreement audits into the reward loop, not just inherit them from the training-data pipeline.

A cross-schema probe. Porting the recipe — three layers, projection-boundary engineering, verifier-gated self-harvest — to an established text-to-SQL benchmark such as BIRD is the direct answer to the single-domain limitation. It tests whether the gain decomposition and the yield-surpassing self-harvest are schema-portable, on a benchmark this dissertation’s authors did not build.

The bootstrap convergence curve. The three completed rounds — teacher harvest, first self-harvest, second self-harvest — sketch a curve whose yield keeps rising while its training gain stops. Two follow-ups suggest themselves. The first is to characterise the curve properly: does a third round’s yield also translate to nothing, and does the point where gains stop move with model scale? The second is to test additive bridge re-consolidation, the lever the convergence analysis identified: raising the export cap on the bucket where the r2 harvest improved most, rather than adding rounds.

Other procurement corpora. The knowledge graph is built from OCDS-standard contract data, and the UK is one publisher among many. Rebuilding the graph over another country’s OCDS releases would hold the schema family fixed while varying the data distribution, conventions, and language of the surfaces. This is a cheaper, intermediate generality test than a full schema change.

The movable projection boundary as a research object. The longer-term agenda turns the dissertation’s mechanism into a question. For a given domain, which semantic properties can be projected into deterministic checks, at what engineering cost, and how does learned competence scale with the projected fraction? This work provides one measured data point — a domain where partial projection plus abstention plus verifier-gated bootstrapping was enough to overtake the cloud teacher — and the v1→v2.2 iteration shows the boundary moves under engineering effort, in both directions. Treating partial verifiability as a general bootstrap principle, testable across domains where verification is inherently incomplete (data quality, code, scientific claims), turns the one-sentence thesis from a finding to be restated into a design law to be tested.

64 Closing

The dissertation opened with a dilemma: the fluent guesser cannot be trusted with public data, and the mute executor cannot be asked a question. The answer defended here is neither to trust the language model nor to abandon it. Instead, we build the seam between the two out of checks wherever a check can reach, out of abstention wherever it cannot, and then let the checked region teach. The checks never verified everything, and they were never claimed to. They verified enough. The competence learned from that partial, filtered, deterministic signal reached beyond the region that produced it. Supervision flows only from the verifiable region, yet the learned competence extends beyond it.

References

- Mohammad Gheshlaghi Azar, Zhaohan Daniel Guo, Bilal Piot, Remi Munos, Mark Rowland, Michal Valko, and Daniele Calandriello. A general theoretical paradigm to understand learning from human preferences. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2024.
- Shulin Cao, Jiaxin Shi, Liangming Pan, Lunyiu Nie, Yutong Xiang, Lei Hou, Juanzi Li, Bin He, and Hanwang Zhang. KQA Pro: A dataset with explicit compositional programs for complex question answering over knowledge base. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023.
- Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. RAFT: Reward ranked finetuning for generative foundation model alignment. *Transactions on Machine Learning Research*, 2023.
- Mohnish Dubey, Debayan Banerjee, Abdelrahman Abdelkawi, and Jens Lehmann. LC-QuAD 2.0: A large dataset for complex question answering over wikidata and DBpedia. In *International Semantic Web Conference (ISWC)*, 2019.
- Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations (ICLR)*, 2024.
- Yu Gu, Xiang Deng, and Yu Su. Don’t generate, discriminate: A proposal for grounding language models to real-world environments. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. Reinforced self-training (ReST) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024a.
- Xiang Huang, Sitao Cheng, Shanshan Huang, Jiayu Shen, Yong Xu, Chaoyun Zhang, and Yuzhong Qu. QueryAgent: A reliable and efficient reasoning framework with environmental feedback based self-correction. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024b.
- Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Wayne Xin Zhao, and Ji-Rong Wen. StructGPT: A general framework for large language model to reason over structured data. *Proceedings of EMNLP*, 2023.
- Amita Kamath, Robin Jia, and Percy Liang. Selective question answering under domain shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

-
- Haoran Luo, Haihong E, Zichen Tang, Shiyao Peng, Yikai Guo, Wentai Zhang, Chenghao Ma, Guanting Dong, Meina Song, Wei Lin, et al. ChatKBQA: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models. *Findings of the Association for Computational Linguistics (ACL)*, 2024a.
- Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations (ICLR)*, 2024b.
- Arka Pal, Deep Karkhanis, Samuel Dooley, Manley Roberts, Siddhartha Naidu, and Colin White. Smaug: Fixing failure modes of preference optimisation with DPO-positive. *arXiv preprint arXiv:2402.13228*, 2024.
- Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don’t know: Unanswerable questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018.
- Avi Singh, John D. Co-Reyes, Rishabh Agarwal, et al. Beyond human data: Scaling self-training for problem-solving in language models. *Transactions on Machine Learning Research*, 2024.
- Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Linzheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, and Zhoujun Li. MAC-SQL: A multi-agent collaborative framework for text-to-SQL. *arXiv preprint arXiv:2312.11242*, 2024.
- Chenglong Wang, Kedar Tatwawadi, Marc Brockschmidt, Po-Sen Huang, Yi Mao, Oleksandr Polozov, and Rishabh Singh. Robust text-to-SQL generation with execution-guided decoding. *arXiv preprint arXiv:1807.03100*, 2018.
- Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

A Reproducibility and number-provenance audit

Every number cited in this paper traces to a single master results table, in which each entry carries its dataset, metric, pipeline version, and an artifact path on disk. Headline figures (the five-system scoreboard, the development-set ladders, the teacher noise floor, the decay figures, and the schema diagnostic) were machine-verified against that table: a numeric-token comparison across all chapters confirmed identical values, precision, and dataset labels. A full per-sentence provenance audit of secondary figures is still in progress.

Instrument verification discipline. Three consecutive, opposite-signed flaws were found in the gates of our own schema diagnostic (an extractor that over-counted failures, a shape gate that over-counted successes, and a content gate that under-counted the untrained baseline), each caught by opening raw outputs rather than trusting the aggregate rate. The project therefore adopted a standing rule: any rate-type, gate-decided number must have ten deterministically-sampled raw decisions (passes and failures) inspected by a human before it enters a table or figure. A suspiciously clean number is treated as an instrument fault until its raw outputs have been read.

Table 10: Resolution of the eight provenance markers left by the drafting pass. All eight figures verified in their primary sources; none required correction.

Figure	Primary source	Disposition
215,221 contract records	KG parquet (recount)	promoted
131,502 canonical organisations	KG parquet (recount)	promoted
99.88% dual-oracle agreement (14,752/14,770)	audit log	promoted
9,762/9,762 LLM-checker approvals	legacy analysis	promoted (motivation only)
Bridge stratum: 86.7% verified vs 52.3% correct	harvest traces	promoted
0.429 median trigram-Jaccard	legacy analysis	promoted (distributional)
166,277 OCDS releases	analysis document	doc-sourced (interim file deleted)
GB-FTS identifier fragmentation survey	analysis document	doc-sourced